==========
=
BCSE329P - BLOCKCHAIN AND DISTRIBUTED LEDGER TECHNOLOGY LAB
MOCK TEST GUIDE - EXACT STEPS FOR ALL ASSIGNMENTS
==========
=

##########
ASSIGNMENT 1
##########

Q1. Blockchain-based Digital Certificate Verification System (Python)

EXACT QUESTION:
"Develop a blockchain-based Digital Certificate Verification System
using Python. The system should create a blockchain where each block
stores the details of a student's digital degree certificate,
including the Certificate ID, Student Name, Degree, and Year of
Graduation. The application should create a Genesis Block, allow new
certificate records to be added as subsequent blocks, display the
complete blockchain, and verify the integrity of the blockchain after
every operation. Additionally, implement a search functionality to
retrieve certificate details using the Certificate ID and incorporate
a tamper detection mechanism that identifies any modification to an
existing certificate record by validating the blockchain hashes. If
any block is altered, the system should display an appropriate
message indicating that the blockchain has been compromised."

STEP 1: Open terminal, create the file
  nano certificate_blockchain.py

STEP 2: Paste the following code exactly

```
  import hashlib
  import datetime

  # ---------------------- Block Class ----------------------
  class Block:
      def __init__(self, index, timestamp, certificate_id,
                   student_name, degree, year, previous_hash):
          self.index = index
          self.timestamp = timestamp
          self.certificate_id = certificate_id
          self.student_name = student_name
          self.degree = degree
          self.year = year
          self.previous_hash = previous_hash
          self.hash = self.calculate_hash()

      def calculate_hash(self):
```

```
        data = (str(self.index) +
                str(self.timestamp) +
                str(self.certificate_id) +
                self.student_name +
                self.degree +
                str(self.year) +
                self.previous_hash)
        return hashlib.sha256(data.encode()).hexdigest()

# ---------------------- Blockchain Class ----------------------
class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]

    def create_genesis_block(self):
        return Block(
            0,
            str(datetime.datetime.now()),
            "GENESIS",
            "Genesis Block",
            "-",
            0,
            "0"
        )

    def get_latest_block(self):
        return self.chain[-1]

    def add_certificate(self, certificate_id, student_name, degree, year):
        previous = self.get_latest_block()
        block = Block(
            len(self.chain),
            str(datetime.datetime.now()),
            certificate_id,
            student_name,
            degree,
            year,
            previous.hash
        )
        self.chain.append(block)
        print("\nCertificate Added Successfully!\n")

    def display_chain(self):
        print("\n========== BLOCKCHAIN ==========\n")
        for block in self.chain:
            print("Block Number :", block.index)
            print("Timestamp :", block.timestamp)
            print("Certificate ID :", block.certificate_id)
            print("Student Name :", block.student_name)
            print("Degree :", block.degree)
            print("Graduation Year :", block.year)
            print("Previous Hash :", block.previous_hash)
```

```
            print("Current Hash :", block.hash)
            print("-" * 60)

    def verify_chain(self):
        for i in range(1, len(self.chain)):
            current = self.chain[i]
            previous = self.chain[i - 1]
            if current.hash != current.calculate_hash():
                return False
            if current.previous_hash != previous.hash:
                return False
        return True

    def search_certificate(self, certificate_id):
        for block in self.chain:
            if block.certificate_id == certificate_id:
                print("\nCertificate Found\n")
                print("Student :", block.student_name)
                print("Degree :", block.degree)
                print("Year :", block.year)
                return
        print("\nCertificate Not Found")

    def tamper_detection(self):
        if self.verify_chain():
            print("\nBlockchain is Secure.")
        else:
            print("\nBlockchain Compromised!")

# ---------------------- Main Program ----------------------
bc = Blockchain()

while True:
    print("\n===== DIGITAL CERTIFICATE BLOCKCHAIN =====")
    print("1. Add Certificate")
    print("2. Display Blockchain")
    print("3. Search Certificate")
    print("4. Verify Blockchain")
    print("5. Tamper Blockchain")
    print("6. Exit")

    choice = int(input("Enter Choice : "))

    if choice == 1:
        cid = input("Certificate ID : ")
        name = input("Student Name : ")
        degree = input("Degree : ")
        year = int(input("Graduation Year : "))
        bc.add_certificate(cid, name, degree, year)

    elif choice == 2:
        bc.display_chain()
```

```
        elif choice == 3:
            cid = input("Enter Certificate ID : ")
            bc.search_certificate(cid)

        elif choice == 4:
            if bc.verify_chain():
                print("\nBlockchain Verified Successfully")
            else:
                print("\nBlockchain Verification Failed")

        elif choice == 5:
            print("\nTampering First Certificate...\n")
            if len(bc.chain) > 1:
                bc.chain[1].student_name = "Hacker"
            bc.tamper_detection()

        elif choice == 6:
            print("Exiting...")
            break

        else:
            print("Invalid Choice")
```

STEP 3: Save and exit nano (Ctrl+O, Enter, Ctrl+X)

STEP 4: Run the program
```
python3 certificate_blockchain.py
```

STEP 5: Demo flow for evaluator
1 -> Add Certificate (e.g. CERT101 / Kushagra Dixit / B.Tech CSE / 2027)
1 -> Add another certificate (CERT102 / another name)
2 -> Display Blockchain (shows Genesis + both blocks with hashes)
3 -> Search Certificate -> enter CERT102 -> shows details
4 -> Verify Blockchain -> "Blockchain Verified Successfully"
5 -> Tamper Blockchain -> changes block[1] student name to "Hacker",
then shows "Blockchain Compromised!"
4 -> Verify Blockchain again -> "Blockchain Verification Failed"
6 -> Exit

KEY POINTS TO EXPLAIN IF ASKED:
- Each block's hash = SHA-256(index+timestamp+certificate_id+name+degree+year+previous_hash)
- Genesis block has previous_hash "0" and certificate_id "GENESIS"
- Tampering changes a stored field but NOT the stored hash, so recalculating the hash on verify no longer matches -> integrity fails.

Q2. Solidity Smart Contract - DigitalIdentity (Remix VM Cancun)

EXACT QUESTION:

"Develop a Solidity smart contract named DigitalIdentity for a blockchain-based digital identity management system. The smart contract should automatically register the Ethereum wallet address of the user invoking the registration function (msg.sender) along with the User ID, Name, and Department. The contract must validate that the User ID is greater than zero and prevent duplicate registrations using the same Ethereum wallet address. It should maintain the total number of successfully registered users and provide functions to retrieve the registered user details and verify whether a wallet has already been registered. Deploy the smart contract using Remix VM (Cancun) and demonstrate its functionality by registering users from at least three different Ethereum accounts, validating duplicate registrations, and displaying the stored user information."

STEP 1: Open https://remix.ethereum.org

STEP 2: In File Explorer, create a new file: Ques2.sol

STEP 3: Paste the following code exactly

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract DigitalIdentity {

    struct User {
        uint userId;
        string name;
        string department;
        address wallet;
        bool registered;
    }

    // Mapping to store user details against wallet address
    mapping(address => User) private users;

    // Total registered users
    uint public totalUsers;

    // Register a new user
    function registerUser(
        uint _userId,
        string memory _name,
        string memory _department
    ) public {

        require(_userId > 0, "User ID must be greater than zero.");
        require(
            users[msg.sender].registered == false,
            "Wallet already registered."
        );
```

```
            users[msg.sender] = User({
                userId: _userId,
                name: _name,
                department: _department,
                wallet: msg.sender,
                registered: true
            });

            totalUsers++;
        }

        // Retrieve details of the caller
        function getUserDetails()
            public
            view
            returns (
                uint,
                string memory,
                string memory,
                address,
                bool
            )
        {
            User memory u = users[msg.sender];
            return (
                u.userId,
                u.name,
                u.department,
                u.wallet,
                u.registered
            );
        }

        // Check whether a wallet is registered
        function isRegistered(address wallet)
            public
            view
            returns (bool)
        {
            return users[wallet].registered;
        }
    }
```

STEP 4: Go to "Solidity Compiler" tab (left sidebar)
- Compiler version should auto-match ^0.8.20 (pick any 0.8.x)
- Click "Compile Ques2.sol"
- Confirm green checkmark / "Compiled" status

STEP 5: Go to "Deploy & Run Transactions" tab
- Environment dropdown -> select "Remix VM (Cancun)"
- Contract dropdown -> select "DigitalIdentity"
- Click "Deploy"

- Contract appears under "Deployed Contracts"

STEP 6: Register user from Account 1 (default selected account)
- Expand the deployed contract -> expand "registerUser"
- Enter: uint256 _userId = 101, string _name = "Kushagra",
  string _department = "CSE"
- Click "transact"
- Check console log: "Transaction mined and execution completed",
  status = true, decoded input shows the values passed

STEP 7: Switch account and register a second user
- Top "Account" dropdown -> select a different address (Account 2)
- Call registerUser again with a different _userId (e.g. 102),
  name, department
- Click "transact" -> success

STEP 8: Register a third user
- Switch to Account 3 in the Account dropdown
- Call registerUser with _userId = 103 and details -> "transact"

STEP 9: Validate duplicate registration
- Stay on Account 3 (already registered)
- Call registerUser again with any values
- Transaction should revert with:
  "Wallet already registered." (shown in the console error)

STEP 10: Validate userId = 0 check
- Switch to a fresh unused account
- Call registerUser with _userId = 0
- Transaction should revert with:
  "User ID must be greater than zero."

STEP 11: Read functions
- getUserDetails() -> click "call" while an already-registered
  account is selected -> returns (userId, name, department,
  wallet address, registered=true)
- isRegistered(address) -> paste any registered wallet address
  as input -> "call" -> returns true
- isRegistered(address) with an unregistered address -> returns false
- totalUsers (public variable, has its own "call" button) -> shows
  the count of successfully registered users (should equal number
  of unique wallets successfully registered, e.g. 3)

KEY POINTS TO EXPLAIN IF ASKED:
- msg.sender is used directly, so the wallet doesn't need to be typed
  in manually during registration.
- mapping(address => User) gives O(1) lookup per wallet.
- require() reverts the whole transaction and refunds unused gas if
  the condition fails.

##

ASSIGNMENT 2
##

Q1. Genesis Block Metadata Analysis (Geth Dev Mode)

EXACT QUESTION:
"Start Geth in Development Mode and investigate the Genesis Block
without executing any Ether transfer transaction. Display the Genesis
Block, node information, network ID, and the number of connected
peers. Retrieve the Genesis Block details and identify its block hash,
parent hash, gas limit, gas used, timestamp, and transaction count.
Finally, explain why the Genesis Block always contains zero
transactions and how it differs from other blocks in the blockchain.
Do not use eth.getBlock("latest") during this experiment."

STEP 1: Start Geth in dev mode
   geth --dev console

STEP 2: Inside the Geth JS console, check node information
   admin.nodeInfo

   (Note down: enode, id, ports, protocols.eth.config, network=1337)

STEP 3: Check the network ID
   net.version
   (Output: "1337")

STEP 4: Check number of connected peers
   net.peerCount
   (Output: 0 -- dev mode has networking disabled)

STEP 5: Retrieve the Genesis Block explicitly by number
   eth.getBlock(0)

   (Do NOT use eth.getBlock("latest") for this experiment)

STEP 6: From the output, identify and note:
   - hash   (block hash of genesis)
   - parentHash   (all zeros: 0x000...000)
   - gasLimit   (e.g. 11500000)
   - gasUsed   (0)
   - timestamp   (0)
   - transactions   ([] -> zero transactions)

STEP 7: Exit console when done
   (Ctrl+D or type: exit)

EXPLANATION TO GIVE:
- The Genesis Block always has zero transactions because it is the
  very first block, created at chain initialization before any
  transaction has ever been broadcast or mined. It exists purely to

define the initial state (pre-funded accounts, chain config) and has no predecessor, hence parentHash = 0x000...0.
- It differs from other blocks because: parentHash is all zeros (every other block's parentHash points to a real previous block hash), it has no miner reward transaction, gasUsed = 0, and its block number is fixed at 0. Every subsequent block links back through previous_hash to this block, forming the chain.

Q2. Transaction Execution Using Gas Estimation (Geth Dev Mode)

EXACT QUESTION:
"Start Geth in Development Mode and use eth.estimateGas() to determine the gas required to transfer 2 ETH from the default Ethereum account to itself. Execute the transaction using a gas limit lower than the estimated value and record the error message generated by Geth. Without modifying the sender account, receiver account, or Ether value, repeat the transaction using the exact estimated gas value. Retrieve the transaction details, transaction receipt, and the block containing the successful transaction. Finally, compare the estimated gas with the actual gas used in the transaction receipt and explain the reason for any difference."

STEP 1: Start Geth in dev mode (if not already running)

```
geth --dev console
```

STEP 2: Estimate gas required for the transfer

```
eth.estimateGas({
    from: eth.accounts[0],
    to: eth.accounts[0],
    value: web3.toWei(2, "ether")
})
```

Output: 21000

STEP 3: Attempt transaction with a LOWER gas limit than estimated

```
eth.sendTransaction({
    from: eth.accounts[0],
    to: eth.accounts[0],
    value: web3.toWei(2, "ether"),
    gas: 20000
})
```

Expected error (record this exactly):
Error: intrinsic gas too low: gas 20000, minimum needed 21000

STEP 4: Repeat the SAME transaction using the exact estimated gas (same sender, same receiver, same value -- only gas changes)

```
eth.sendTransaction({
    from: eth.accounts[0],
    to: eth.accounts[0],
```

```
    value: web3.toWei(2, "ether"),
    gas: 21000
  })
```

Output: transaction hash, e.g.
"0x967b64ad426d2902444f2852237c21838f65cb3d5dd529020145d38119e71aca"

STEP 5: Retrieve transaction details (use your actual tx hash)

```
eth.getTransaction("<paste_tx_hash_here>")
```

Note: gas: 21000, gasPrice, from, to, value, nonce, blockNumber

STEP 6: Retrieve the transaction receipt

```
eth.getTransactionReceipt("<paste_tx_hash_here>")
```

Note: status: "0x1" (success), gasUsed: 21000,
cumulativeGasUsed: 21000, blockNumber

STEP 7: Retrieve the block containing this transaction

```
eth.getBlock(eth.blockNumber)
```

Note: gasUsed, transactions array containing your tx hash,
miner address, timestamp

EXPLANATION TO GIVE:
- estimateGas() returned 21000, and the actual gasUsed in the receipt was also exactly 21000.
- A plain ETH transfer (no contract code, no calldata) always costs exactly the base intrinsic gas of 21000 -- this is a fixed protocol constant (21000 gas per basic transaction), so the estimate and the actual usage match exactly with no difference. If this had been a contract call, estimated and actual gas could differ slightly due to conditional logic (if/require branches) not all being executed during actual execution.

##
ASSIGNMENT 3
##

Q1. Simplified Bitcoin Miner Using Rust

EXACT QUESTION:
"Develop a simplified bitcoin miner using rust."

STEP 1: Check Rust and Cargo installation

```
rustc --version
cargo --version
```

STEP 2: (Only if Rust is NOT installed) Install via rustup

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

```
   -- choose option 1 (Proceed with standard installation)
   source "$HOME/.cargo/env"

STEP 3: Create a new Rust project
   cargo new bitcoin_miner
   cd bitcoin_miner

STEP 4: Add the SHA-256 dependency
   cargo add sha2

STEP 5: Write the miner
   nano src/main.rs

   use sha2::{Digest, Sha256};
   use std::time::Instant;

   struct Block {
      previous_hash: String,
      data: String,
      timestamp: u64,
      difficulty: usize,
      nonce: u64,
   }

   impl Block {
      fn calculate_hash(&self) -> String {
         let input = format!(
            "{}{}{}{}",
            self.previous_hash,
            self.data,
            self.timestamp,
            self.nonce
         );

         let mut hasher = Sha256::new();
         hasher.update(input.as_bytes());

         let result = hasher.finalize();

         result.iter().map(|b| format!("{:02x}", b)).collect()
      }

      fn mine(&mut self) -> (String, u64, u64) {
         let target = "0".repeat(self.difficulty);
         let start = Instant::now();

         loop {
            let hash = self.calculate_hash();

            if hash.starts_with(&target) {
               let elapsed = start.elapsed().as_millis();
               return (hash, self.nonce, elapsed as u64);
```

```
            }

            self.nonce += 1;
        }
    }
}

fn main() {
    println!("=== Simplified Bitcoin Miner ===");
    println!();

    let mut block = Block {
        previous_hash:
            "00000000000000000000123456789abcdef".to_string(),

        data:
            "Alice pays Bob 1 BTC".to_string(),

        timestamp:
            1_725_000_000,

        difficulty:
            4,

        nonce:
            0,
    };

    println!("Mining block...");
    println!("Difficulty: {} leading zeros", block.difficulty);
    println!("Transaction: {}", block.data);
    println!();

    let (hash, nonce, elapsed) = block.mine();

    println!("Block mined successfully!");
    println!();
    println!("Nonce: {}", nonce);
    println!("Hash: {}", hash);
    println!("Time taken: {} ms", elapsed);
}
```

STEP 6: Check the program compiles

```
cargo check
```

(NOTE: if you see an error formatting the hash type with {:x},
make sure you're using:

```
result.iter().map(|b| format!("{:02x}", b)).collect()
```

and NOT: format!("{:x}", result))

STEP 7: Run the miner

```
cargo run
```

Expected output pattern:
=== Simplified Bitcoin Miner ===
Mining block...
Difficulty: 4 leading zeros
Transaction: Alice pays Bob 1 BTC
Block mined successfully!
Nonce: <some number>
Hash: 0000<...>
Time taken: <n> ms

STEP 8: Verify Proof-of-Work
Confirm the printed Hash starts with "0000" (4 leading zeros,
matching difficulty = 4).

PROJECT STRUCTURE:

```
bitcoin_miner/
├── Cargo.toml
├── Cargo.lock
└── src/
    └── main.rs
```

Q2. Deploy and Simulate a Local Private Blockchain Network Using Rust

---

EXACT QUESTION:
"Deploy and simulate a local private blockchain network using rust."

STEP 1: Verify Rust installation

```
rustc --version
cargo --version
```

STEP 2: Create the Rust project

```
cargo new private_blockchain
cd private_blockchain
```

STEP 3: Add dependencies

```
nano Cargo.toml
```

```
[package]
name = "private_blockchain"
version = "0.1.0"
edition = "2024"

[dependencies]
tokio = { version = "1", features = ["full"] }
sha2 = "0.10"
serde = { version = "1", features = ["derive"] }
serde_json = "1"
chrono = "0.4"
```

STEP 4: Build to confirm dependencies resolve

```
cargo build
```

STEP 5: Create the module files

```
mkdir -p src
touch src/block.rs
nano src/block.rs
nano src/blockchain.rs
nano src/network.rs
nano src/main.rs
```

STEP 6: src/block.rs -- Block structure

```
use chrono::Utc;
use serde::{Deserialize, Serialize};
use sha2::{Digest, Sha256};

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Transaction {
    pub sender: String,
    pub receiver: String,
    pub amount: f64,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Block {
    pub index: u64,
    pub timestamp: String,
    pub transactions: Vec<Transaction>,
    pub previous_hash: String,
    pub hash: String,
    pub nonce: u64,
}

impl Block {
    pub fn new(
        index: u64,
        transactions: Vec<Transaction>,
        previous_hash: String,
    ) -> Self {
        let timestamp = Utc::now().to_rfc3339();

        let mut block = Block {
            index,
            timestamp,
            transactions,
            previous_hash,
            hash: String::new(),
            nonce: 0,
        };

        block.mine();
```

```
        block
    }

    pub fn calculate_hash(&self) -> String {
        let data = format!(
            "{}{}{:?}{}{}",
            self.index,
            self.timestamp,
            self.transactions,
            self.previous_hash,
            self.nonce
        );

        let mut hasher = Sha256::new();
        hasher.update(data.as_bytes());

        format!("{:x}", hasher.finalize())
    }

    pub fn mine(&mut self) {
        let difficulty = 4;

        loop {
            let hash = self.calculate_hash();

            if hash.starts_with(&"0".repeat(difficulty)) {
                self.hash = hash;
                break;
            }

            self.nonce += 1;
        }
    }
}
```

STEP 7: src/blockchain.rs -- Blockchain

```
use crate::block::{Block, Transaction};

pub struct Blockchain {
    pub chain: Vec<Block>,
}

impl Blockchain {
    pub fn new() -> Self {
        let genesis_transaction = Transaction {
            sender: "System".to_string(),
            receiver: "Network".to_string(),
            amount: 0.0,
        };

        let genesis_block =
```

```
            Block::new(0, vec![genesis_transaction], "0".to_string());

        Blockchain {
            chain: vec![genesis_block],
        }
    }

    pub fn add_block(&mut self, transactions: Vec<Transaction>) {
        let previous_block = self.chain.last().unwrap();

        let new_block = Block::new(
            self.chain.len() as u64,
            transactions,
            previous_block.hash.clone(),
        );

        self.chain.push(new_block);
    }

    pub fn is_valid(&self) -> bool {
        for i in 1..self.chain.len() {
            let current = &self.chain[i];
            let previous = &self.chain[i - 1];

            if current.hash != current.calculate_hash() {
                return false;
            }

            if current.previous_hash != previous.hash {
                return false;
            }
        }

        true
    }

    pub fn print_chain(&self) {
        println!("\n========== BLOCKCHAIN ==========");

        for block in &self.chain {
            println!("\nBlock #{}", block.index);
            println!("Timestamp    : {}", block.timestamp);
            println!("Transactions : {:?}", block.transactions);
            println!("Previous Hash: {}", block.previous_hash);
            println!("Hash         : {}", block.hash);
            println!("Nonce        : {}", block.nonce);
        }

        println!("\nBlockchain valid: {}", self.is_valid());
        println!("================================\n");
    }
}
```

STEP 8: src/network.rs -- Node / private network simulation

```
use crate::block::{Block, Transaction};
use crate::blockchain::Blockchain;

pub struct Node {
    pub id: String,
    pub blockchain: Blockchain,
}

impl Node {
    pub fn new(id: &str) -> Self {
        Node {
            id: id.to_string(),
            blockchain: Blockchain::new(),
        }
    }

    pub fn add_block(&mut self, transactions: Vec<Transaction>) {
        self.blockchain.add_block(transactions);
    }

    pub fn receive_block(&mut self, block: Block) {
        let expected_index = self.blockchain.chain.len() as u64;

        if block.index != expected_index {
            println!("{} rejected block: invalid index", self.id);
            return;
        }

        let previous_hash = &self.blockchain.chain.last().unwrap().hash;

        if block.previous_hash != *previous_hash {
            println!("{} rejected block: previous hash mismatch", self.id);
            return;
        }

        if block.hash != block.calculate_hash() {
            println!("{} rejected block: invalid hash", self.id);
            return;
        }

        self.blockchain.chain.push(block);

        println!("{} accepted the new block", self.id);
    }

    pub fn print_status(&self) {
        println!(
            "{} -> {} blocks | Valid: {}",
            self.id,
```

```
            self.blockchain.chain.len(),
            self.blockchain.is_valid()
        );
    }
}
```

STEP 9: src/main.rs -- main program

```
mod block;
mod blockchain;
mod network;

use block::Transaction;
use network::Node;

fn main() {
    println!("========================================");
    println!("     PRIVATE BLOCKCHAIN NETWORK");
    println!("========================================");

    println!("\nCreating three private blockchain nodes...");

    let mut node1 = Node::new("Node-1");
    let mut node2 = Node::new("Node-2");
    let mut node3 = Node::new("Node-3");

    println!("Node-1 created");
    println!("Node-2 created");
    println!("Node-3 created");

    println!("\nAll nodes contain the Genesis Block.");

    node1.print_status();
    node2.print_status();
    node3.print_status();

    println!("\n========== MINING BLOCK 1 ==========");

    let transactions = vec![
        Transaction {
            sender: "Alice".to_string(),
            receiver: "Bob".to_string(),
            amount: 50.0,
        },
        Transaction {
            sender: "Bob".to_string(),
            receiver: "Charlie".to_string(),
            amount: 20.0,
        },
    ];

    node1.add_block(transactions);
```

```
let block1 = node1.blockchain.chain.last().unwrap().clone();

println!("\nNode-1 mined Block #{}", block1.index);
println!("Hash : {}", block1.hash);
println!("Nonce: {}", block1.nonce);

println!("\nBroadcasting Block #1 to other nodes...");

node2.receive_block(block1.clone());
node3.receive_block(block1.clone());

println!("\nNetwork status:");

node1.print_status();
node2.print_status();
node3.print_status();

println!("\n========== MINING BLOCK 2 ==========");

let transactions2 = vec![
    Transaction {
        sender: "Charlie".to_string(),
        receiver: "Alice".to_string(),
        amount: 15.0,
    },
];

node1.add_block(transactions2);

let block2 = node1.blockchain.chain.last().unwrap().clone();

println!("\nNode-1 mined Block #{}", block2.index);
println!("Hash : {}", block2.hash);
println!("Nonce: {}", block2.nonce);

println!("\nBroadcasting Block #2...");

node2.receive_block(block2.clone());
node3.receive_block(block2.clone());

println!("\nFinal network status:");

node1.print_status();
node2.print_status();
node3.print_status();

println!("\n========== BLOCKCHAIN DATA ==========");

node1.blockchain.print_chain();

println!("\n========== TAMPER TEST ==========");
```

```
    node1.blockchain.chain[1].transactions[0].amount = 9999.0;

    println!("Block #1 transaction modified!");

    println!(
        "Blockchain valid after tampering: {}",
        node1.blockchain.is_valid()
    );

    println!("\nPrivate blockchain simulation completed.");
}
```

STEP 10: Check the project structure

```
tree
```

Expected:

```
private_blockchain/
├── Cargo.lock
├── Cargo.toml
├── src
│   ├── block.rs
│   ├── blockchain.rs
│   ├── main.rs
│   └── network.rs
└── target
```

STEP 11: Compile

```
cargo build
```

STEP 12: Run

```
cargo run
```

Expected final status lines:

```
Node-1 -> 3 blocks | Valid: true
Node-2 -> 3 blocks | Valid: true
Node-3 -> 3 blocks | Valid: true
...
Blockchain valid after tampering: false
```

##
ASSIGNMENT 4
##

Q1. Blockchain with 3 blocks - Tamper Detection in Rust

EXACT QUESTION:
"Create a blockchain containing three blocks. After creating the blockchain, modify the transaction data in Block 1. Write a Rust program to detect and report the tampering."

STEP 1: Create the project

```
cargo new blockchain_tampering
cd blockchain_tampering
```

STEP 2: Add the SHA-256 dependency

```
nano Cargo.toml

[dependencies]
sha2 = "0.10"
```

STEP 3: Write src/main.rs

```
nano src/main.rs

use sha2::{Digest, Sha256};

struct Block {
    index: u32,
    data: String,
    previous_hash: String,
    hash: String,
}

fn calculate_hash(index: u32, data: &str, previous_hash: &str) -> String {
    let input = format!("{}{}{}", index, data, previous_hash);

    let mut hasher = Sha256::new();
    hasher.update(input.as_bytes());

    format!("{:x}", hasher.finalize())
}

fn verify_blockchain(blockchain: &[Block]) {
    for i in 0..blockchain.len() {
        let block = &blockchain[i];

        let calculated_hash =
            calculate_hash(block.index, &block.data, &block.previous_hash);

        if calculated_hash != block.hash {
            println!("Block {}: Tampering detected", block.index);
            return;
        }

        if i > 0 && block.previous_hash != blockchain[i - 1].hash {
            println!("Block {}: Tampering detected", block.index);
            return;
        }

        println!("Block {}: Valid", block.index);
    }
```

```
    println!("Blockchain is valid");
  }

  fn main() {
    let hash0 = calculate_hash(0, "Alice pays Bob 100", "0");

    let block0 = Block {
      index: 0,
      data: String::from("Alice pays Bob 100"),
      previous_hash: String::from("0"),
      hash: hash0,
    };

    let hash1 = calculate_hash(1, "Bob pays Charlie 50", &block0.hash);

    let block1 = Block {
      index: 1,
      data: String::from("Bob pays Charlie 50"),
      previous_hash: block0.hash.clone(),
      hash: hash1,
    };

    let hash2 = calculate_hash(2, "Charlie pays David 20", &block1.hash);

    let block2 = Block {
      index: 2,
      data: String::from("Charlie pays David 20"),
      previous_hash: block1.hash.clone(),
      hash: hash2,
    };

    let mut blockchain = vec![block0, block1, block2];

    println!("Before tampering:");
    verify_blockchain(&blockchain);

    blockchain[1].data = String::from("Bob pays Charlie 500");

    println!("\nAfter tampering:");
    verify_blockchain(&blockchain);
  }
```

STEP 4: Run

```
  cargo run

  Expected output:
  Before tampering:
  Block 0: Valid
  Block 1: Valid
  Block 2: Valid
  Blockchain is valid
```

After tampering:
Block 0: Valid
Block 1: Tampering detected

EXPLANATION TO GIVE:
- Block 1's data was changed from "Bob pays Charlie 50" to
  "Bob pays Charlie 500" but its stored hash was NOT updated.
- Recalculating the hash from the new data gives a different value
  than the stored hash -> mismatch -> "Tampering detected".
- Block 2 is never checked because the loop returns as soon as the
  first tampered block is found.

Q2. Merkle Tree from 4 Transactions in Rust

EXACT QUESTION:
"Write a Rust program to construct a Merkle Tree from four blockchain transactions and calculate the Merkle Root. Modify one transaction and demonstrate that the Merkle Root changes."

STEP 1: Create the project
```
cargo new merkle_tree
cd merkle_tree
```

STEP 2: Add the SHA-256 dependency
```
nano Cargo.toml

[dependencies]
sha2 = "0.10"
```

STEP 3: Write src/main.rs
```
nano src/main.rs

use sha2::{Digest, Sha256};

fn hash(data: &str) -> String {
    let mut hasher = Sha256::new();
    hasher.update(data.as_bytes());
    format!("{:x}", hasher.finalize())
}

fn merkle_root(transactions: &[String]) -> String {
    let mut hashes: Vec<String> =
        transactions.iter().map(|t| hash(t)).collect();

    while hashes.len() > 1 {
        let mut next_level = Vec::new();

        for i in (0..hashes.len()).step_by(2) {
            let combined = format!("{}{}", hashes[i], hashes[i + 1]);
            next_level.push(hash(&combined));
```

```
            }

            hashes = next_level;
        }

        hashes[0].clone()
    }

    fn main() {
        let mut transactions = vec![
            String::from("Alice pays Bob 100"),
            String::from("Bob pays Charlie 50"),
            String::from("Charlie pays David 20"),
            String::from("David pays Alice 30"),
        ];

        let root1 = merkle_root(&transactions);

        println!("Merkle Root before modification:");
        println!("{}", root1);

        transactions[1] = String::from("Bob pays Charlie 500");

        let root2 = merkle_root(&transactions);

        println!("\nMerkle Root after modification:");
        println!("{}", root2);

        if root1 != root2 {
            println!("\nTampering detected: Merkle Root changed");
        }
    }
```

STEP 4: Run

```
cargo run

Expected output pattern:
Merkle Root before modification:
<hash>

Merkle Root after modification:
<different hash>

Tampering detected: Merkle Root changed
```

EXPLANATION TO GIVE:

- All 4 transactions are hashed individually (leaf hashes).
- Leaf hashes are combined in pairs and re-hashed, level by level, until only one hash remains -- the Merkle Root.
- Changing even one transaction changes its leaf hash, which propagates upward and changes every parent hash above it, including the final Merkle Root. This is how a Merkle Tree lets

you detect tampering in any single transaction without re-checking
every transaction individually -- you just compare the root.

==========
QUICK COMMAND CHEAT-SHEET (in case you just need the commands fast)
==========

Rust projects:
```
cargo new <project_name>
cd <project_name>
cargo add sha2              (or add manually to Cargo.toml)
nano src/main.rs            (paste code, Ctrl+O Enter Ctrl+X)
cargo check                 (fast syntax/type check)
cargo build                 (compile)
cargo run                   (compile + run)
```

Python:
```
nano <file>.py
python3 <file>.py
```

Geth dev mode:
```
geth --dev console
admin.nodeInfo
net.version
net.peerCount
eth.getBlock(0)
eth.estimateGas({from:.., to:.., value: web3.toWei(<n>, "ether")})
eth.sendTransaction({from:.., to:.., value:.., gas: <n>})
eth.getTransaction("<hash>")
eth.getTransactionReceipt("<hash>")
eth.getBlock(eth.blockNumber)
```

Remix (Solidity):
```
remix.ethereum.org -> new file -> paste code
Solidity Compiler tab -> Compile
Deploy & Run tab -> Environment: Remix VM (Cancun) -> Deploy
Switch "Account" dropdown to test multiple wallets
Expand function -> fill params -> transact / call
```

==========
END OF GUIDE
==========

==========
APPENDED SUBMISSIONS - LAB SUBMISSION 5 AND LAB SUBMISSION 6
(Converted from the PDF files into the format used above)

==========
=

Notes on conversion: PDF screenshots cannot be embedded in a .txt file,
so each screenshot is described under a [SCREENSHOT] tag, using only
what is visible in it. Anything small, truncated or not clearly legible
is marked [FLAG: ...] instead of being guessed. Code, commands,
identifiers and addresses are copied exactly; only line breaks /
indentation were re-flowed where the PDF text extraction had scrambled
them.

##########
ASSIGNMENT 5
##########

Q1. ERC-20 Token (BlessToken) - Ganache (Local) and MetaMask (Testnet)

SUBMISSION DETAILS (from PDF cover page):
Title : Blockchain and Distributed Ledger Technology Lab
Assessment : 5
Name : Kushagra Dixit
Reg No : 23BKT0121

EXACT QUESTION:
"Develop and deploy an ERC-20 token using Solidity and demonstrate its
deployment in the following two environments:"

a) Ganache - Local Blockchain:
Deploy the ERC-20 token on a local Ganache blockchain using Remix.
Demonstrate a token transfer between two Ganache accounts and verify
the transaction.

b) MetaMask - Ethereum Testnet:
Connect Remix with MetaMask and deploy the same ERC-20 token on an
Ethereum testnet. Perform a token transfer using MetaMask and verify
the transaction.

EXPECTED SUBMISSION:
1. ERC-20 Solidity smart contract code.
2. Screenshots showing deployment using Ganache.
3. Screenshot showing successful token transfer in Ganache.
4. Screenshots showing deployment using MetaMask on an Ethereum
testnet.
5. Screenshot showing successful token transfer using MetaMask.
6. Deployed contract address and transaction hash for both
environments.

STEP 1: ERC-20 Solidity smart contract code (PDF section "Code")
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

```
contract MyToken {
    string public name = "BlessToken";
    string public symbol = "BLT";
    uint8 public decimals = 18;
    uint256 public totalSupply;

    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;

    event Transfer(address indexed from, address indexed to, uint256 value);
    event Approval(address indexed owner, address indexed spender, uint256 value);

    constructor(uint256 initialSupply) {
        totalSupply = initialSupply * 10 ** decimals;
        balanceOf[msg.sender] = totalSupply;
        emit Transfer(address(0), msg.sender, totalSupply);
    }

    function transfer(address to, uint256 value) public returns (bool) {
        require(balanceOf[msg.sender] >= value, "Insufficient balance");
        balanceOf[msg.sender] -= value;
        balanceOf[to] += value;
        emit Transfer(msg.sender, to, value);
        return true;
    }

    function approve(address spender, uint256 value) public returns (bool) {
        allowance[msg.sender][spender] = value;
        emit Approval(msg.sender, spender, value);
        return true;
    }

    function transferFrom(address from, address to, uint256 value) public returns (bool) {
        require(balanceOf[from] >= value, "Insufficient balance");
        require(allowance[from][msg.sender] >= value, "Allowance exceeded");
        allowance[from][msg.sender] -= value;
        balanceOf[from] -= value;
        balanceOf[to] += value;
        emit Transfer(from, to, value);
        return true;
    }
}
```

STEP 2: Ganache (PDF section "Ganache")

[SCREENSHOT - Ganache CLI terminal output]

- Header: ganache v7.9.2 (@ganache/cli: 0.10.2, @ganache/core: 0.10.2) / Starting RPC server
- Available Accounts: (0) to (9), each showing (1000 ETH)
- Account (0): 0x214Ac48c6Ea335EAfba8B4E469bdEaaF5Dbb351D
- Account (1): 0x3458F1e6641Ae13F3631f79F502bb539ADccC424
- Private Keys: (0) to (9) listed

- HD Wallet: Mnemonic listed; Base HD Path: m/44'/60'/0'/0/{account_index}
- Default Gas Price: 2000000000
- BlockGas Limit: 30000000
- Call Gas Limit: 50000000
- Chain - Hardfork: shanghai, Id: 1337
- RPC Listening on 127.0.0.1:7545

[FLAG: Addresses of accounts (2)-(9), all 10 private keys and the mnemonic are visible in the screenshot but were NOT transcribed (small print; a single wrong character would make them wrong, and they are wallet credentials). Accounts (0) and (1) are transcribed because the same addresses also appear in the Remix transfer and balanceOf screenshots. Mixed-case (checksum) letters were read from the image - verify against the PDF.]

STEP 3: Remix - Ganache deployment and transfer (PDF section "Remix")

[SCREENSHOT 1 - Remix, Deploy & Run Transactions, Remix VM]

- Environment: Remix VM, with a second dropdown showing "Osaka"; Account 1 with 100.000 ETH
- File MyToken.sol open (same code as Step 1); deploy panel shows MyToken - MyToken.sol, "Compiled", and an initialSupply (uint256) input

[FLAG: This screenshot is Remix VM (not Ganache); the PDF gives no caption for it. Account address is truncated.]

[SCREENSHOT 2 - Remix connected to Ganache, contract deployed]

- Browser address bar shows remix.ethereum.org with compiler string soljson-v0.8.34+commit.80d5c536.js
- Environment: Dev - Ganache Provider; Account 16 (0x214...b351d), balance 999.998 ETH
- Deployed Contracts: 1 - MyToken, labelled Ganache Testnet, at 0x3d4...9121d, with its functions listed (approve, transfer, ...)
- Console: [block:1 txIndex:0] from: 0x214...b351D to: MyToken.(constructor) value: 0 wei logs: 1
- Console: [Verification] Contract deployed. Checking explorers for registration...
- Console: Etherscan verification skipped: API key not provided. Please input the API key in Remix Settings - Connected Services OR Contract Verification Plugin Settings.
- Console: [Sourcify] Contract not found on Sourcify after 30s. Explorer indexing timed out.

[FLAG: Compiler string and the hash/data fields of the console line are small or truncated in the screenshot; treat as approximate.]

[SCREENSHOT 3 - Remix Deploy panel (Ganache) + deployment details]

- Environment: Dev - Ganache Provider; Account 16 (0x214...b351d), 1000.000 ETH (before deployment)
- Deploy button label: Ganache Testnet (1337); contract MyToken - MyToken.sol, Compiled
- "Verify Contract on Explorers" toggle is switched on; initialSupply (uint256) input; Value 0 wei; Gas limit field

[FLAG: The value typed into initialSupply is not visible in the screenshot (only the placeholder 'uint256').]
Deployment details shown as terminal text:
Transaction: 0x563d597232b60e4a95288398c24ed558b00b5fcb31e09d4ce48e4b20eecbfe1f
Contract created: 0x3d4daf6923841b01abe4b9622d53dae5ed39121d
Gas usage: 935273
Block number: 1
Block time: Mon Sep 21 2026 21:12:13 GMT+0530 (India Standard Time)

[SCREENSHOT 4 - Token transfer on Ganache (transfer function)]
- Environment: Dev - Ganache Provider; Account 16, balance 999.998 ETH
- transfer (address, uint256) filled with: 0x3458f1e6641ae13f3631f79f502bb539adccc424 and 1000000000000000000
- Console: transfer transaction mined - [block:2 txIndex:0] from: 0x214...b351D to: MyToken.transfer(address,uint256) 0x3d4...9121d value: 0 wei logs: 1

[FLAG: Block/txIndex and the transaction hash of the transfer (appears as 0x6bb...0b904 or similar) are truncated and small; the full transfer hash is NOT legible.]

[SCREENSHOT 5 - Verifying the transfer with balanceOf]
- balanceOf (address) input: 0x3458f1e6641ae13f3631f79f502bb539adccc424
- Result: 0: uint256: 1000000000000000000
- Console: [call] from: 0x214ac48c6ea335eafba8b4e469bdeaaf5dbb351d to: MyToken.balanceOf(address)
- Functions visible below: decimals, name, symbol, totalSupply (each with a Call button)

Ganache summary (items 2, 3 and 6 of the expected submission):
- Deployed contract address (Ganache): 0x3d4daf6923841b01abe4b9622d53dae5ed39121d
- Deployment transaction hash (Ganache): 0x563d597232b60e4a95288398c24ed558b00b5fcb31e09d4ce48e4b20eecbfe1f
- Transfer: 1000000000000000000 sent from Account 0x214Ac48c...b351D to 0x3458F1e6...cC424; balanceOf of the receiver returns 1000000000000000000

STEP 4: MetaMask - Ethereum Sepolia testnet (PDF section "Metamask")
[SCREENSHOT 1 - Remix with MetaMask, before deployment]
- Remix banner: "Remix is moving to app.remix.live. Your Workspaces are stored in the browser and won't be accessible from the new URL unless you follow the move instructions."
- Environment: Browser Extension, Sepolia Testnet; Account 26 (0xF23...2dD71), 0.050 ETH
- Deploy button label: Sepolia (11155111); contract MyToken - MyToken.sol; "Verify Contract on Explorers" on; Value 0 wei
- MetaMask popup: Account 1 (0xF23c6...2dD71), $0.00, Network: Sepolia, SepoliaETH balance 0.0500

[SCREENSHOT 2 - Contract deployed on Sepolia]
- Account 26 balance: 0.047 ETH; Deployed Contracts: 1 - MyToken on Sepolia (0x6CE...646b8, truncated)
- Console: transaction from 0xF23...2dD71 to: MyToken.(constructor) value: 0 wei logs: 1, with 'view on Etherscan' and 'view on Blockscout' links
- Console: [Verification] Contract deployed. Checking explorers for registration...
- MetaMask: SepoliaETH balance 0.0476

[FLAG: The block number of the deployment ([block:11752467 txIndex:7] as read) and the deployment transaction hash are truncated/small in the screenshot; the full Sepolia DEPLOYMENT transaction hash is NOT legible anywhere in the PDF.]

[SCREENSHOT 3 - Token transfer from Remix via MetaMask]
- transfer (address, uint256) filled with recipient 0x91333077C45a... (field truncated) and 1000000000000000000
- Console: transaction from 0xF23...2dD71 to: MyToken.transfer(address,uint256) 0x6CEE...646b8 value: 0 wei logs: 1 ([block:11752482 txIndex:6] as read), with Etherscan / Blockscout links
- Console: a verification message ending "Polling timed out. Please open the Contract Verification plugin to check details."
- Account 26 balance: 0.047 ETH; MetaMask SepoliaETH balance 0.0474

[SCREENSHOT 4 - Sepolia Etherscan transaction page (verification)]
- Banner: [This is a Sepolia Testnet transaction only]
- Transaction Action: 0xF23c68aB...a21c2dD71 Transfer 1 BLT to 0x91333077...5a041679d
- Tabs: Overview, Logs (1), State
- Transaction Hash: 0x5b7edaf955aae17160a48688ee9b1eeedaf7052a1a13dd12636d65ced6f62e50
- Status: Success
- Block: 11752482 (2 Block Confirmations)
- Timestamp: 46 secs ago, Sep-21-2026 05:54:24 PM +UTC
- From: 0xF23c68aB7cFa2F227046bdEbB525196a21c2dD71
- Interacted With (To): 0x6CEE254AeaeBcA2372cf48DE96e1d3bFc10646b8
- ERC-20 Tokens Transferred: From 0xF23c68aB...a21c2dD71 To 0x91333077...5a041679d For 1 ERC-20: BlessToken (BLT)

[FLAG: The lower part of the page (Value, Transaction Fee, etc.) is hidden behind a cookie banner in the screenshot.]

MetaMask / Sepolia summary (items 4, 5 and 6 of the expected submission):
- Deployed contract address (Sepolia): 0x6CEE254AeaeBcA2372cf48DE96e1d3bFc10646b8
- Token transfer transaction hash (Sepolia): 0x5b7edaf955aae17160a48688ee9b1eeedaf7052a1a13dd12636d65ced6f62e50
- Deployment transaction hash (Sepolia): not legible in the PDF

(see flag above)

FLAGS FOR SUBMISSION 5 (unreadable / ambiguous items):
- Ganache accounts (2)-(9), private keys and mnemonic: present in screenshot, deliberately not transcribed.
- initialSupply value used at deployment: not visible.
- Ganache transfer transaction hash: truncated; Sepolia deployment transaction hash: truncated.
- Recipient address on Sepolia: only 0x91333077...5a041679d (Etherscan) / 0x91333077C45a... (Remix) visible; full address not legible.
- Small console text (block numbers, compiler string, Remix version label) read approximately.

##
ASSIGNMENT 6
##

SUBMISSION DETAILS (from PDF cover page):
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
Fall Semester 2025-2026
BCSE329P - Blockchain and Distributed Ledger Technology Lab
Digital Assignment - DA-6
Name : Kushagra Dixit
Registration Number : 23BKT0121
Course : BCSE329P - Blockchain and Distributed Ledger Technology Lab
Assignment : DA-6
Submission Date : 27/09/2026

Q1. Hyperledger Fabric Using Docker

EXACT QUESTION:
"A consortium of two organizations wants to develop a permissioned blockchain-based asset-tracking system using Hyperledger Fabric. Study and explain how the Fabric network can be designed and deployed using Docker. Draw the network architecture and describe the roles of organizations, peers, ordering service, Certificate Authority, channels, ledger, and chaincode. Explain the complete procedure and important commands for creating the custom network, creating a channel, deploying chaincode, and performing transactions. Actual implementation is not required."

SOLUTION:
Hyperledger Fabric is a permissioned blockchain framework, which fits this scenario well because every participant (in this case, the two consortium organizations) must be identified through a Membership Service Provider (MSP) before they can read or write to the ledger. Unlike public blockchains, nobody can join anonymously, and the two organizations can still keep their transactions restricted to a private

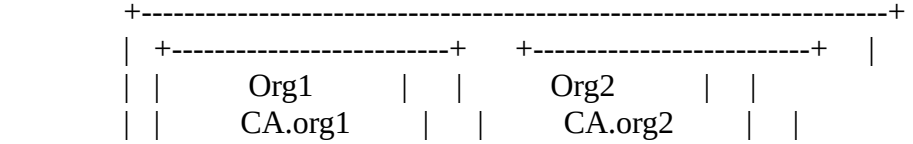
communication channel between themselves. Each Fabric component (peers, orderer, Certificate Authorities, CouchDB) runs as a separate Docker container, and Docker Compose is used to bring the whole network up on a single host for development and testing.

STEP 1: Network Architecture

For a two-organization consortium, the network consists of the following components:

- Organizations (Org1, Org2): Each organization represents one member of the consortium (for example, a manufacturer and a distributor in the asset-tracking use case). Each org has its own Membership Service Provider (MSP) that issues and validates the digital identities of its peers, admin, and client applications.
- Certificate Authority (CA): Each organization runs its own Fabric CA server (ca.org1, ca.org2). The CA issues X.509 certificates to peers, orderers, and application clients belonging to that org, which is what the MSP uses to verify identity and role during endorsement and validation.
- Peers: Each org runs one or more peer nodes (peer0.org1, peer0.org2). Peers hold a copy of the ledger, execute (simulate) chaincode when endorsing a transaction proposal, and validate/commit blocks received from the orderer.
- Ordering Service: A separate orderer node (or a small Raft cluster of orderers, for crash-fault tolerance) collects endorsed transactions from clients, orders them into blocks, and distributes the blocks to all peers on the channel. The orderer does not execute chaincode and does not hold application state - it only sequences transactions.
- Channel: A channel (for example, assetchannel) is a private communication path joining Org1's and Org2's peers together. Each channel has its own separate ledger, so if more organizations join later without needing to see this data, a separate channel can be created for them instead.
- Ledger: The ledger on each peer has two parts - the blockchain, which is an append-only, hash-linked chain of blocks recording every transaction, and the world state, which stores only the current value of each asset key (typically in a CouchDB or LevelDB instance running alongside the peer) so that queries do not need to replay the whole chain.
- Chaincode: The smart contract (for example, assetTracking) contains the business logic - functions such as CreateAsset, TransferAsset, and ReadAsset. It is installed on the peers of both organizations and is governed by an endorsement policy (for example, "an endorsement from Org1 AND Org2 is required") before a transaction can be committed to the ledger.

STEP 2: Simple Network Diagram



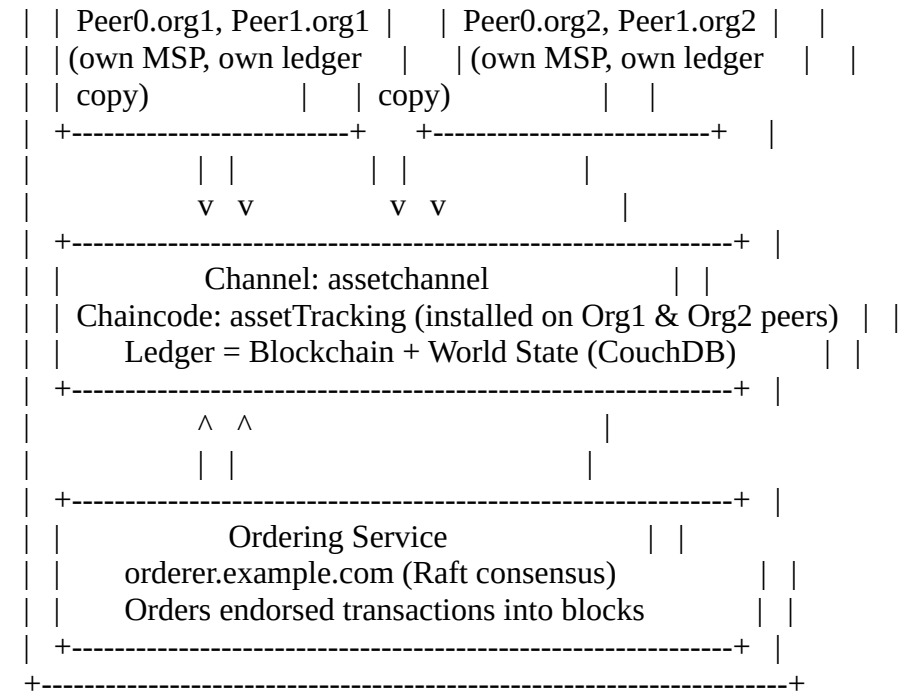


Figure 1: Simplified two-organization Fabric network architecture
[FLAG: The PDF diagram has one down-arrow pair (Org boxes -> Channel) and one up-arrow pair (Ordering Service -> Channel); the ASCII layout above redraws it with the same box texts. Exact arrow positions are approximate.]

STEP 3: Docker-Based Deployment

Every Fabric component above runs as an independent Docker container using the standard Hyperledger Fabric images (hyperledger/fabric-peer, hyperledger/fabric-orderer, hyperledger/fabric-ca, couchdb). A single docker-compose.yaml file defines and links all these containers on a common Docker network so that peers can reach the orderer, and clients can reach the peers, using container hostnames. Before the containers are started, the organizations' cryptographic identities and the channel's genesis configuration must be generated:

- crypto-config.yaml describes the two organizations, their peers, and their admin/user identities. Running cryptogen generate --config=crypto-config.yaml produces the certificates and keys (MSP folders) for every peer, orderer, and admin.
- configtx.yaml defines the consortium (Org1 + Org2), the orderer profile, and the application channel profile. It is used to generate the genesis block for the orderer and the channel creation transaction.

STEP 4: Procedure and Important Commands

4.1 Generate crypto material and channel artifacts

```
cryptogen generate --config=./crypto-config.yaml

configtxgen -profile TwoOrgsOrdererGenesis \
   -channelID system-channel -outputBlock ./channel-artifacts/genesis.block
```

```
configtxgen -profile TwoOrgsChannel \
  -outputCreateChannelTx ./channel-artifacts/assetchannel.tx \
  -channelID assetchannel
```

4.2 Bring up the network containers

```
docker-compose -f docker-compose.yaml up -d
```

This starts ca.org1, ca.org2, orderer.example.com, peer0.org1, peer0.org2, their CouchDB instances, and a Fabric CLI container used to issue peer commands.

4.3 Create and join the channel

```
peer channel create -o orderer.example.com:7050 -c assetchannel \
  -f ./channel-artifacts/assetchannel.tx --tls --cafile $ORDERER_CA

# run once with CORE_PEER_* env vars set for Org1's peer0
peer channel join -b assetchannel.block

# run again with CORE_PEER_* env vars set for Org2's peer0
peer channel join -b assetchannel.block
```

4.4 Package, install, approve and commit the chaincode

```
peer lifecycle chaincode package assetTracking.tar.gz \
  --path ./chaincode/assetTracking --lang node --label assetTracking_1

# installed separately on Org1's peer and Org2's peer
peer lifecycle chaincode install assetTracking.tar.gz

peer lifecycle chaincode approveformyorg -o orderer.example.com:7050 \
  --channelID assetchannel --name assetTracking --version 1.0 \
  --package-id assetTracking_1:<hash> --sequence 1 --tls --cafile $ORDERER_CA

peer lifecycle chaincode checkcommitreadiness \
  --channelID assetchannel --name assetTracking --version 1.0 --sequence 1

peer lifecycle chaincode commit -o orderer.example.com:7050 \
  --channelID assetchannel --name assetTracking --version 1.0 --sequence 1 \
  --peerAddresses peer0.org1.example.com:7051 \
  --peerAddresses peer0.org2.example.com:9051 --tls --cafile $ORDERER_CA
```

4.5 Perform transactions

```
peer chaincode invoke -o orderer.example.com:7050 -C assetchannel -n assetTracking \
  --peerAddresses peer0.org1.example.com:7051 \
  --peerAddresses peer0.org2.example.com:9051 --tls --cafile $ORDERER_CA \
  -c '{"function":"CreateAsset","Args":["Asset1","Widget","Org1","1000"]}'

peer chaincode query -C assetchannel -n assetTracking \
  -c '{"function":"ReadAsset","Args":["Asset1"]}'
```

Once committed, every subsequent CreateAsset / TransferAsset invocation must be endorsed by peers from both organizations (as per the endorsement policy) before the orderer packages it into a block and every peer commits it to its own copy of the ledger. As per the

question, this design and the above command sequence are explained conceptually; the network was not actually stood up for this assignment.

[FLAG: In the PDF the command blocks of Steps 4.1-4.5 were scrambled in text extraction; they were re-ordered from the page layout. Placement of the '# run once / # run again' and '# installed separately' comment lines directly above their commands is reconstructed. The step numbering 4.1-4.5 corresponds to the PDF's numbered headings 1-5.]

Q2. Hyperledger Fabric Blockchain Service on AWS Cloud

EXACT QUESTION:
"An organization wants to migrate its Hyperledger Fabric asset-tracking network from a local environment to AWS Cloud. Study and design an AWS-based Hyperledger Fabric architecture using suitable AWS services such as Amazon EC2, VPC, Security Groups, IAM, EBS, and CloudWatch. Explain how Fabric components can be deployed using Docker on AWS, how the network can be secured and monitored, and how chaincode and transactions can be handled. Finally, compare the local Docker-based deployment with the AWS Cloud deployment in terms of security, scalability, availability, cost, and management. Actual AWS deployment is not required."

SOLUTION:
The local Docker Compose setup from Q1 works well for development, but it runs every container on a single machine, which is a single point of failure and cannot be scaled beyond that machine's resources. Moving the same network to AWS lets each organization's Fabric components run on dedicated, independently manageable cloud infrastructure while keeping the underlying Fabric/Docker setup almost identical. The design below reuses the same Docker images and Fabric commands from Q1, but places them on AWS infrastructure.

STEP 1: Proposed AWS Architecture

- VPC (Virtual Private Cloud): One VPC per organization (or a shared VPC with per-org subnets, connected via VPC peering / a Transit Gateway if the orgs are on separate AWS accounts). Each VPC has a public subnet (for a bastion host / load balancer) and private subnets (for peers, orderer, and CA), so that consensus and ledger traffic is never directly reachable from the internet.
- EC2 instances: Each Fabric component (CA, peer, orderer, CouchDB) runs inside a Docker container hosted on its own EC2 instance (or a small number of instances running Docker Compose/Docker Swarm services), mirroring the local setup but with one physical machine replaced by one or more EC2 instances per org, placed in private subnets.
- Security Groups: Fine-grained inbound rules per component - the peer's Security Group allows inbound traffic on 7051 (peer endpoint) and 7052 (chaincode) only from the other org's peer/orderer Security Groups, not from 0.0.0.0/0; the orderer's

Security Group allows 7050 only from peer Security Groups; the CA's Security Group allows 7054 only from the admin/bastion Security Group; SSH (22) is restricted to the bastion host only.

- IAM: IAM roles attached to EC2 instances follow least privilege - for example, permission to read crypto material from a specific S3 bucket and to write logs to CloudWatch, nothing broader. Human administrators use named IAM users/roles with MFA rather than shared root credentials.
- EBS (Elastic Block Store): Each peer and orderer EC2 instance has a dedicated EBS volume for its ledger data (blockchain files, CouchDB state database, and the orderer's Raft log). Volumes are encrypted at rest and snapshotted on a schedule for backup/disaster recovery.
- CloudWatch: The CloudWatch agent on each EC2 instance ships Docker container logs (peer, orderer, CA logs) to CloudWatch Logs, and CloudWatch Metrics/Alarms watch CPU, memory, and disk usage of each instance plus the health of each Fabric container, so an admin is alerted if a peer or the orderer goes down.

STEP 2: Deploying Fabric with Docker on AWS

The container images and Fabric commands are unchanged from the local setup in Q1 - the same docker-compose.yaml pattern (or Docker Swarm service definitions, if components are spread across multiple EC2 instances) is used to start ca.org1, ca.org2, orderer.example.com, and each org's peers. The only things that change are: (1) crypto material (MSP certificates from cryptogen or a Fabric CA server) is generated once and securely copied to each EC2 instance, for example via an encrypted S3 bucket rather than being baked into a public AMI; and (2) the CORE_PEER_ADDRESS / orderer address values in each container's environment point at the private DNS names or IP addresses of the AWS-hosted instances instead of localhost. Channel creation, chaincode packaging/install/approve/commit, and transaction invoke/query all use the exact same peer channel, peer lifecycle chaincode, and peer chaincode invoke/query commands shown in Q1, run from a CLI container or admin machine that has network access into the VPC (typically over a VPN or through the bastion host).

STEP 3: Security and Monitoring

Security on AWS is layered on top of Fabric's own TLS and MSP-based identity: VPC subnetting and Security Groups stop unauthorized network access before a request even reaches a Fabric container; IAM restricts what each instance and each admin can do at the AWS API level; EBS and S3 encryption protect data at rest; and admin access uses the bastion host or AWS Systems Manager Session Manager instead of broadly open SSH. Monitoring relies on CloudWatch Logs for the Fabric peer/orderer/CA logs, CloudWatch Alarms for instance and container health, and (optionally) CloudTrail to audit any change made to the AWS infrastructure itself (who modified a Security Group, launched or stopped an instance, and so on).

STEP 4: Comparison: Local Docker Deployment vs AWS Cloud Deployment

SECURITY:
- Local Docker (Q1): Relies only on the host OS and Docker's own network isolation; no separation between organizations at the infrastructure level since everything runs on one machine.
- AWS Cloud (Q2): VPC subnetting, per-component Security Groups, IAM least privilege, and encrypted EBS/S3 give network- and access-level isolation between organizations, in addition to Fabric's own TLS/MSP security.

SCALABILITY:
- Local Docker (Q1): Limited to the CPU/RAM of the one machine running Docker Compose; adding peers means adding more containers on the same host.
- AWS Cloud (Q2): Elastic - more EC2 instances (and therefore more peers, or a larger Raft orderer cluster) can be added on demand as the network grows.

AVAILABILITY:
- Local Docker (Q1): Single point of failure; if the host machine goes down, the entire network (all orgs) goes down with it.
- AWS Cloud (Q2): Instances can be spread across multiple Availability Zones, EBS snapshots support recovery, and a multi-node Raft orderer can tolerate the loss of a node without the network going offline.

COST:
- Local Docker (Q1): No cloud cost - only the electricity/hardware of the local machine, but it does not reflect any real running cost of a production network.
- AWS Cloud (Q2): Pay-as-you-go cost for EC2, EBS, data transfer, and CloudWatch; needs ongoing cost monitoring (e.g. AWS Budgets), but the cost is directly tied to actual usage and can be scaled down when not needed.

MANAGEMENT:
- Local Docker (Q1): Simple for a single developer - one docker-compose up command brings up the whole network; not realistic for two genuinely independent organizations.
- AWS Cloud (Q2): Requires more operational setup (ideally Infrastructure-as-Code such as CloudFormation/Terraform) and more moving parts, but it lets each organization actually own, secure, and bill for its own infrastructure independently - which is closer to how a real multi-organization consortium would be run.

As with Q1, this AWS architecture is presented as a proposed design; no AWS resources were actually provisioned for this assignment.

Q3. Course Project: First Functional Module Implementation

EXACT QUESTION:

"As part of the course project, identify and implement the first functional module/component of your proposed project."

"Submit approximately 30-40% of the overall project implementation as source code, focusing on the first major functional module/component of your project. Along with the source code, provide a brief description explaining:"

- What the implemented module represents in your project.
- The purpose and functionality of the implemented module.
- How the module contributes to the overall project.
- How the submitted code works and the main functions/components implemented.
- The technologies and tools used for implementing the module.
- The blockchain platform/network, wallet, smart contract and other relevant blockchain components involved, wherever applicable.
- How blockchain is incorporated into the proposed solution.
- The data/dataset/input data used for implementing and testing the module, wherever applicable.
- The initial setup/configuration required to run the implemented module.

"Demonstrate the working status of the implemented portion, wherever applicable, and provide relevant inputs, outputs, screenshots, transactions and/or test results."

"Focus: Setup and initial implementation of the first functional module."

SOLUTION:

STEP 1: Project Overview

The proposed course project is TraceLedger, a blockchain-based IoT supply chain monitoring system. It tracks a shipment as it moves through a supply chain and immutably records on a blockchain the moment a shipment is damaged in transit - because of a temperature excursion, excessive humidity, or excessive vibration/shock - along with the GPS location where the breach occurred. Company admins use a dashboard to see every shipment's status and, for any breached shipment, exactly where in the journey it broke. The full project is built in phases: (1) the smart contract, (2) a backend API with a device simulator, (3) an admin dashboard, (4) testnet deployment, and (5) real ESP32 hardware integration (not started yet, since the hardware is not available). This submission focuses on Phase 1 - the smart contract, which is the first functional module of the project and the on-chain foundation that every later phase builds on.

STEP 2: What the Module Represents and Its Purpose

This module is the ShipmentRegistry smart contract - the single source of truth for the project. It represents the blockchain layer of TraceLedger: the part of the system that actually makes the

shipment records tamper-evident. Its job is to (a) let a company admin register a new shipment along with the acceptable temperature/humidity/vibration thresholds for that shipment, (b) let an authorized device account permanently record a breach event whenever a sensor reading crosses one of those thresholds, and (c) let an admin mark a shipment as delivered. Every one of these actions is recorded on-chain as an event, so the history cannot be edited afterwards.

STEP 3: Contribution to the Overall Project

All the later phases of TraceLedger are built around this contract rather than around a database. The backend (Phase 2) is a thin layer that calls into this contract using ethers.js whenever a shipment is created or a breach is detected, and it keeps a Postgres cache purely for fast dashboard queries - the cache is always re-derivable from the contract's events, never the other way round. The dashboard (Phase 3) reads shipment status, breach history, and transaction hashes that ultimately come from this contract. Without this module there is nothing for the rest of the project to record data onto, which is why it was implemented first.

STEP 4: How the Submitted Code Works - Main Functions

The contract is written in Solidity and uses OpenZeppelin's AccessControl for role-based permissions. Two roles are defined: ADMIN_ROLE (can create shipments and mark them delivered) and DEVICE_ROLE (can record a breach). The main functions are:

- createShipment(...) - registers a new shipment with its product, origin, destination, and threshold values; restricted to ADMIN_ROLE; rejects a shipment ID that already exists and rejects invalid threshold ranges (min > max); emits ShipmentCreated.
- recordBreach(...) - restricted to DEVICE_ROLE; stores a breach (type, measured value, GPS latitude/longitude, timestamp, device ID) against a shipment, flips its status to Breached, and refuses to record a breach on a shipment that has already been delivered; emits BreachRecorded.
- markDelivered(...) - restricted to ADMIN_ROLE; sets a shipment's status to Delivered; emits ShipmentDelivered.
- getShipment(...) / getBreaches(...) / getShipmentIds() - read-only view functions used by the backend to display shipment and breach data.

All numeric sensor values (temperature, humidity, vibration, GPS coordinates) are stored as scaled integers (for example, a temperature of 22.4°C is stored as 2240) because Solidity has no native decimal type; the scaling/unscaling is handled in the backend's blockchain service, shown below.

STEP 5: Source Code

File: contracts/contracts/ShipmentRegistry.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;
```

```
import "@openzeppelin/contracts/access/AccessControl.sol";

/// @notice Records supply-chain shipments and immutable breach events
/// (temperature, humidity, vibration) with GPS location, on-chain.
contract ShipmentRegistry is AccessControl {
    bytes32 public constant ADMIN_ROLE = keccak256("ADMIN_ROLE");
    bytes32 public constant DEVICE_ROLE = keccak256("DEVICE_ROLE");

    enum BreachType { Temperature, Humidity, Vibration }
    enum ShipmentStatus { InTransit, Breached, Delivered }

    struct Thresholds {
        int256 tempMin;      // degrees C, scaled by 100
        int256 tempMax;
        uint256 humidityMin; // percent, scaled by 100
        uint256 humidityMax;
        uint256 vibrationMax; // sensor units, scaled by 100
    }

    struct Shipment {
        string product;
        string origin;
        string destination;
        Thresholds thresholds;
        ShipmentStatus status;
        bool exists;
    }

    struct Breach {
        BreachType breachType;
        int256 measuredValue;
        int256 lat; // degrees, scaled by 1e6
        int256 lon;
        uint256 timestamp;
        string deviceId;
    }

    mapping(string => Shipment) private shipments;
    mapping(string => Breach[]) private shipmentBreaches;
    string[] private shipmentIds;

    event ShipmentCreated(string indexed shipmentId, string product, string origin, string
destination);
    event BreachRecorded(string indexed shipmentId, BreachType breachType, int256
measuredValue, int256 lat, int256 lon, uint256 timestamp, string deviceId);
    event ShipmentDelivered(string indexed shipmentId);

    constructor(address initialAdmin) {
        _grantRole(DEFAULT_ADMIN_ROLE, initialAdmin);
        _grantRole(ADMIN_ROLE, initialAdmin);
    }
```

```
    modifier shipmentExists(string calldata shipmentId) {
        require(shipments[shipmentId].exists, "shipment does not exist");
        _;
    }

    function createShipment(
        string calldata shipmentId, string calldata product, string calldata origin, string calldata destination,
        int256 tempMin, int256 tempMax, uint256 humidityMin, uint256 humidityMax, uint256 vibrationMax
    ) external onlyRole(ADMIN_ROLE) {
        require(!shipments[shipmentId].exists, "shipment already exists");
        require(tempMin <= tempMax, "tempMin must be <= tempMax");
        require(humidityMin <= humidityMax, "humidityMin must be <= humidityMax");

        shipments[shipmentId] = Shipment({
            product: product, origin: origin, destination: destination,
            thresholds: Thresholds(tempMin, tempMax, humidityMin, humidityMax, vibrationMax),
            status: ShipmentStatus.InTransit, exists: true
        });
        shipmentIds.push(shipmentId);
        emit ShipmentCreated(shipmentId, product, origin, destination);
    }

    function recordBreach(
        string calldata shipmentId, BreachType breachType, int256 measuredValue,
        int256 lat, int256 lon, uint256 timestamp, string calldata deviceId
    ) external onlyRole(DEVICE_ROLE) shipmentExists(shipmentId) {
        Shipment storage shipment = shipments[shipmentId];
        require(shipment.status != ShipmentStatus.Delivered, "shipment already delivered");

        shipmentBreaches[shipmentId].push(Breach(breachType, measuredValue, lat, lon, timestamp, deviceId));
        shipment.status = ShipmentStatus.Breached;
        emit BreachRecorded(shipmentId, breachType, measuredValue, lat, lon, timestamp, deviceId);
    }

    function markDelivered(string calldata shipmentId) external onlyRole(ADMIN_ROLE) shipmentExists(shipmentId) {
        shipments[shipmentId].status = ShipmentStatus.Delivered;
        emit ShipmentDelivered(shipmentId);
    }

    function getShipment(string calldata shipmentId) external view shipmentExists(shipmentId)
        returns (string memory, string memory, string memory, Thresholds memory, ShipmentStatus) {
        Shipment storage s = shipments[shipmentId];
        return (s.product, s.origin, s.destination, s.thresholds, s.status);
    }
```

```
    function getBreaches(string calldata shipmentId) external view shipmentExists(shipmentId)
returns (Breach[] memory) {
        return shipmentBreaches[shipmentId];
    }

    function getShipmentIds() external view returns (string[] memory) {
        return shipmentIds;
    }
}
```

File: contracts/scripts/deploy.js

```
const hre = require("hardhat");

async function main() {
  const [deployer] = await hre.ethers.getSigners();
  console.log("Deploying ShipmentRegistry with admin:", deployer.address);

  const ShipmentRegistry = await hre.ethers.getContractFactory("ShipmentRegistry");
  const registry = await ShipmentRegistry.deploy(deployer.address);
  await registry.waitForDeployment();

  console.log("ShipmentRegistry deployed to:", await registry.getAddress());
}

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

The backend's blockchain service (Phase 2, included here only as supporting code to show how the module is consumed) wraps the contract with ethers.js, converts real-world decimal values into the contract's scaled integers, and exposes plain JavaScript functions to the rest of the API:

File: backend/src/services/blockchain.js (excerpt)

```
function init() {
  if (contract) return;
  provider = new ethers.JsonRpcProvider(process.env.RPC_URL);
  if (process.env.ADMIN_PRIVATE_KEY) adminWallet = new
    ethers.Wallet(process.env.ADMIN_PRIVATE_KEY, provider);
  if (process.env.DEVICE_PRIVATE_KEY) deviceWallet = new
    ethers.Wallet(process.env.DEVICE_PRIVATE_KEY, provider);

  const address = process.env.CONTRACT_ADDRESS;
  contract = new ethers.Contract(address, artifact.abi, adminWallet ||
    provider);
  deviceContract = new ethers.Contract(address, artifact.abi, deviceWallet
    || provider);
}

async function recordBreach({ shipmentId, breachType, measuredValue, lat,
```

```
    lon, timestamp, deviceId }) {
    init();
    const tx = await deviceContract.recordBreach(
      shipmentId, BREACH_TYPE_INDEX[breachType], Math.round(measuredValue *
      100),
      Math.round(lat * GPS_SCALE), Math.round(lon * GPS_SCALE), timestamp,
      deviceId
    );
    const receipt = await tx.wait();
    return { txHash: receipt.hash, blockNumber: receipt.blockNumber };
  }
```

[FLAG: The three code files were scrambled by text extraction in the PDF (e.g. the tail of getShipment() appears after the start of deploy.js). They were re-ordered and re-indented from the page layout; all tokens, names, strings and comments are unchanged. The string "humidityMin must be <= humidityMax" is wrapped across two lines in the PDF and is joined with a single space here. BREACH_TYPE_INDEX, GPS_SCALE and artifact are used in the blockchain.js excerpt but defined outside the excerpt shown.]

STEP 6: Technologies and Tools Used

- Solidity ^0.8.24 for the smart contract
- OpenZeppelin Contracts (AccessControl) for role-based permissions
- Hardhat 2.x as the development, compilation, testing, and deployment framework (pinned to Hardhat 2 because the development machine runs Node 18, and Hardhat 3 requires Node 22+)
- Hardhat/Chai/Mocha for the automated unit test suite
- Ethers.js v6, used by the backend to call the deployed contract

STEP 7: Blockchain Platform, Smart Contract and Related Components

The blockchain platform used is Ethereum. The module was developed and unit-tested against a local Hardhat network first, and has since been deployed to the public Ethereum Sepolia testnet at contract address 0x0160697212eE63deBF678041768Ef9D7719d6286 as part of the project's later Sepolia-deployment phase. There is no separate end-user wallet UI; instead, the backend holds two role-based Ethereum accounts on the server side - an admin wallet (holds ADMIN_ROLE, used to create shipments and mark them delivered) and a device wallet (holds DEVICE_ROLE, used to record breaches on behalf of the IoT device/simulator). The smart contract itself, ShipmentRegistry, is the only blockchain component in this module; there is no token or NFT involved.

STEP 8: How Blockchain Is Incorporated into the Solution

Only the events that matter for trust and auditability - a shipment being created, a breach occurring, and a shipment being delivered - are written on-chain, since every write costs gas. Continuous sensor telemetry (routine, non-breach temperature/humidity/vibration/GPS readings) is stored only in the off-chain Postgres cache for chart rendering, and is never written to the contract. This keeps the blockchain as the tamper-evident record of exactly the events a

company or an auditor would care about, while keeping gas costs proportional to how many actual breaches occur rather than how often a sensor reports in.

STEP 9: Data Used for Implementation and Testing

No external dataset is required for this module, since it is a smart contract rather than a data-processing component. Testing uses synthetic fixture data defined directly in the Hardhat test file - for example, a sample shipment SHP-1001 ("Vaccines", Mumbai Warehouse -> Delhi Cold Storage) with temperature thresholds of -5.00°C to 8.00°C, humidity thresholds of 20% to 60%, and a vibration threshold of 5.00 units, along with sample breach values used to exercise the threshold logic.

STEP 10: Initial Setup / Configuration to Run This Module

No .env values are required to compile and run the unit tests locally - Hardhat's built-in in-memory network and test accounts are used automatically. An RPC URL, a funded account private key, and a deployed contract address are only needed once the module is connected to a real network (local persistent node or Sepolia), which is handled by the backend's own .env file in later phases.

```
cd contracts
npm install                # installs Hardhat, ethers, OpenZeppelin contracts, chai
npx hardhat compile        # compiles ShipmentRegistry.sol
npx hardhat test           # runs the unit test suite against an in-memory
                           # Hardhat network

# to try it against a persistent local network:
npx hardhat node           # terminal 1, keep running
npx hardhat run scripts/deploy.js --network localhost   # terminal 2
```

[FLAG: The comments beside these commands were scrambled in the PDF text; their attachment to each command is reconstructed from the wording (e.g. '# terminal 1, keep running' on 'npx hardhat node').]

STEP 11: Working Status and Test Results

The contract compiles cleanly and its full unit test suite passes: 11 out of 11 tests, covering role-based access control (only an admin can create a shipment or mark it delivered; only a device account can record a breach) and shipment-lifecycle/threshold validation (rejecting an invalid threshold range, a duplicate shipment ID, operations on a non-existent shipment, breaches after delivery, and correctly tracking multiple shipment IDs). The screenshot below was captured directly from a terminal running these commands on the project machine.

[SCREENSHOT - Figure 2: terminal running npx hardhat compile and npx hardhat test]

- Warning lines: You are currently using Node.js v18.19.1, which is not supported by Hardhat
- Prompts shown: anonymous crash reports / basic usage data (answered y); install Hardhat for Visual Studio Code extension

(answered y)
- "Nothing to compile" followed by "=== running tests ==="
- Test tree: ShipmentRegistry -> access control (5 checks) and shipment lifecycle & threshold validation (6 checks), all ticked
- Result line: 11 passing

Figure 2: npx hardhat compile and npx hardhat test run on the actual project - all 11 unit tests pass
[FLAG: The individual test names in the screenshot are too small to transcribe reliably, so only the group headings, counts and the '11 passing' result are recorded; the list of what the tests cover is taken from the PDF body text above.]

Beyond local testing, this contract has also already been deployed to the public Ethereum Sepolia testnet as part of the project's later deployment phase, and has genuinely processed on-chain transactions from earlier end-to-end testing of the full system (shipment creation and breach-recording calls from the backend). The screenshot below is Etherscan's live public page for the deployed contract address, showing its real transaction history (59 transactions at the time of capture) - this is a read-only, publicly verifiable page and was not created or edited for this report.

[SCREENSHOT - Figure 3: Etherscan page of the deployed contract]
- Sepolia Testnet Etherscan page for Contract 0x0160697212eE63deBF678041768Ef9D7719d6286
- Overview: ETH balance 0 ETH; More Info: contract creator and creation time ("18 days ago" as read)
- Transactions tab: "Latest 25 from a total of 59 transactions" with rows for each call (Method, Block, Age, From, To, Amount, Txn Fee)

Figure 3: ShipmentRegistry contract deployed on Ethereum Sepolia testnet, viewed on Etherscan (address 0x0160697212eE63deBF678041768Ef9D7719d6286)
[FLAG: Transaction hashes, method names, block numbers and the contract creator address in the rows of this screenshot are too small to read reliably and are not transcribed. The '59 transactions' figure is stated in the PDF body text.]

The remaining phases of TraceLedger (backend API, device simulator, admin dashboard, and ESP32 hardware integration) are not part of this submission, which is scoped to the first functional module as required by the question.

FLAGS FOR SUBMISSION 6 (unreadable / ambiguous items):
- Q1 diagram redrawn in ASCII; arrow placement approximate.
- Q1 command blocks and Q3 code files were scrambled in text extraction; re-ordered from page layout with no token changes.
- Q3 Figure 2: individual test names not legible; Figure 3: row-level details not legible.

==========
=
END OF APPENDED SUBMISSIONS (LAB SUBMISSION 5 AND 6)
==========
=

==========
=
APPENDED SUBMISSION - LAB SUBMISSION 7
(Converted from the PDF file into the format used above)
==========
=

Notes on conversion: PDF screenshots cannot be embedded in a .txt file,
so terminal screenshots are transcribed under a [SCREENSHOT] tag.
Anything not clearly legible or inconsistent is marked [FLAG: ...]
instead of being guessed. Code, identifiers and outputs are copied
exactly; only line breaks / indentation were re-flowed where the PDF
text extraction had lost them. Math symbols were converted to ASCII (SUM
for the sigma sign, <=, >=, !=, 2^20).

##########
ASSIGNMENT 7
##########

SUBMISSION DETAILS (from PDF cover page):
  Title   : Blockchain and Distributed Ledger Technology Lab
  Assessment : 7
  Name   : Kushagra Dixit
  Reg No   : 23BKT0121

Q1. Bitcoin Transaction Fee Selection Module (Rust)

EXACT QUESTION:
"Implement a Bitcoin transaction fee selection module using Rust. Given
a set of pending transactions with different transaction fees, develop a
miner module that selects transactions for inclusion in a block based on
their transaction fees. Calculate the total transaction fees and the
final reward received by the miner."

STEP 1: Theory
  - Mempool and transaction fees. When a user broadcasts a Bitcoin
    transaction, it waits in the mempool (the pool of unconfirmed
    transactions) until a miner includes it in a block. The
    transaction fee is not a separate field. It is the difference
    between the total value of the inputs and the total value of the
    outputs (Fee = SUM inputs - SUM outputs). Fees are measured in
    satoshis (1 BTC = 100,000,000 satoshis).

- Limited block space. A block can hold only a limited amount of data (about 1,000,000 virtual bytes in Bitcoin; 1,000 vB in our demo). Because space is scarce, miners cannot include every pending transaction and must choose.

- Fee rate. A large transaction takes more space than a small one, so the absolute fee alone is misleading. Miners compare transactions by fee rate:
  Fee rate = Fee / Size (satoshis per virtual byte, sat/vB)

- Selection strategy. The miner wants to maximize the total fees collected within the size limit. This is a knapsack problem. Since individual transactions are tiny compared to the block, the greedy approach of picking transactions in descending order of fee rate gives near-optimal results and is the standard strategy used in practice. (Bitcoin Core additionally considers transaction packages, such as child-pays-for-parent. This is outside the scope of this module.)

- Miner reward. The miner is paid through the coinbase transaction, whose value is:
  Reward = Block subsidy + Total fees of included transactions
  The block subsidy halves every 210,000 blocks. After the April 2024 halving it is 3.125 BTC (312,500,000 satoshis).

STEP 2: Algorithm

Input : list of pending transactions (id, size, fee), block size limit, block subsidy

Output: selected transactions, total fees, miner reward

1. Compute the fee rate (fee / size) for every transaction in the mempool.
2. Sort the transactions in descending order of fee rate.
3. Initialize used_size = 0, total_fees = 0, and an empty selected list.
4. For each transaction in the sorted list:
   - o If used_size + tx.size <= block_limit, add it to selected, then update used_size += tx.size and total_fees += tx.fee.
   - o Otherwise, skip it (it stays in the mempool).
5. Compute reward = block_subsidy + total_fees.
6. Display the selected transactions, total fees, and the final miner reward.

Time complexity: O(n log n), dominated by the sorting step.

Result for the demo data: Transactions tx3, tx7, tx4 and tx8 were selected (900 / 1000 vB used). Total fees = 29,500 sat. Final reward = 312,500,000 + 29,500 = 312,529,500 sat = 3.12529500 BTC.

STEP 3: Code (Rust)

```
use std::cmp::Ordering;
```

```
/// 1 BTC = 100,000,000 satoshis
const SATS_PER_BTC: u64 = 100_000_000;
/// Block subsidy after the April 2024 halving: 3.125 BTC
const BLOCK_SUBSIDY_SATS: u64 = 312_500_000;
/// Demo block capacity in virtual bytes (real Bitcoin: ~1,000,000 vB)
const MAX_BLOCK_VSIZE: u32 = 1_000;

#[derive(Debug, Clone)]
struct Transaction {
    id: &'static str,
    vsize: u32, // size in virtual bytes
    fee: u64,   // fee in satoshis (inputs - outputs)
}

impl Transaction {
    fn fee_rate(&self) -> f64 {
        self.fee as f64 / self.vsize as f64
    }
}

struct Miner {
    max_block_vsize: u32,
    subsidy: u64,
}

struct BlockTemplate {
    selected: Vec<Transaction>,
    rejected: Vec<Transaction>,
    used_vsize: u32,
    total_fees: u64,
    reward: u64,
}

impl Miner {
    /// Greedy selection: highest fee rate first, skip anything that no longer fits.
    fn select_transactions(&self, mempool: &[Transaction]) -> BlockTemplate {
        let mut sorted = mempool.to_vec();
        sorted.sort_by(|a, b| {
            b.fee_rate()
                .partial_cmp(&a.fee_rate())
                .unwrap_or(Ordering::Equal)
                .then_with(|| b.fee.cmp(&a.fee))
        });

        let mut selected = Vec::new();
        let mut rejected = Vec::new();
        let mut used_vsize = 0u32;
        let mut total_fees = 0u64;

        for tx in sorted {
            if used_vsize + tx.vsize <= self.max_block_vsize {
                used_vsize += tx.vsize;
```

```
                total_fees += tx.fee;
                selected.push(tx);
            } else {
                rejected.push(tx);
            }
        }

        BlockTemplate {
            selected,
            rejected,
            used_vsize,
            total_fees,
            reward: self.subsidy + total_fees,
        }
    }
}

fn btc(sats: u64) -> String {
    format!("{}.{:08} BTC", sats / SATS_PER_BTC, sats % SATS_PER_BTC)
}

fn print_table(title: &str, txs: &[Transaction]) {
    println!("{title}");
    println!("  {:<6} {:>8} {:>12} {:>14}", "TxID", "vsize", "fee (sat)", "rate (sat/vB)");
    for t in txs {
        println!("  {:<6} {:>8} {:>12} {:>14.2}", t.id, t.vsize, t.fee, t.fee_rate());
    }
    println!();
}

fn main() {
    let mempool = vec![
        Transaction { id: "tx1", vsize: 250, fee: 5_000 },
        Transaction { id: "tx2", vsize: 400, fee: 4_000 },
        Transaction { id: "tx3", vsize: 150, fee: 6_000 },
        Transaction { id: "tx4", vsize: 300, fee: 9_000 },
        Transaction { id: "tx5", vsize: 500, fee: 7_500 },
        Transaction { id: "tx6", vsize: 200, fee: 1_000 },
        Transaction { id: "tx7", vsize: 350, fee: 12_000 },
        Transaction { id: "tx8", vsize: 100, fee: 2_500 },
    ];

    let miner = Miner { max_block_vsize: MAX_BLOCK_VSIZE, subsidy:
BLOCK_SUBSIDY_SATS };

    println!("=== Bitcoin Transaction Fee Selection (DA-7 Q1) ===\n");
    println!("Block capacity : {} vB", miner.max_block_vsize);
    println!("Block subsidy  : {}\n", btc(miner.subsidy));

    print_table("Pending transactions (mempool):", &mempool);

    let block = miner.select_transactions(&mempool);
```

```
    print_table("Selected for block (highest fee rate first):", &block.selected);
    print_table("Left in mempool (did not fit):", &block.rejected);

    println!("Block space used       : {} / {} vB", block.used_vsize, miner.max_block_vsize);
    println!("Total transaction fees : {} sat ({})", block.total_fees, btc(block.total_fees));
    println!("Block subsidy          : {} sat ({})", miner.subsidy, btc(miner.subsidy));
    println!(
       "Final miner reward     : {} sat ({})",
       block.reward,
       btc(block.reward)
    );

    // sanity check: reward must equal subsidy + sum of selected fees
    let check: u64 = block.selected.iter().map(|t| t.fee).sum::<u64>() + miner.subsidy;
    assert_eq!(check, block.reward);
    assert!(block.used_vsize <= miner.max_block_vsize);
  }
```

[FLAG: Code indentation was lost in the PDF text and has been reconstructed; all tokens, names and strings are unchanged. Column spacing inside the println! format strings (e.g. 'Block subsidy :') is approximate because the PDF collapsed repeated spaces.]

STEP 4: Output (PDF screenshot of the terminal)

[SCREENSHOT - terminal output]

```
PS C:\Users\kushagra\BDLT_DA5 > cargo run --release --bin q1
=== Bitcoin Transaction Fee Selection (DA-7 Q1) ===

Block capacity : 1000 vB
Block subsidy  : 3.12500000 BTC

Pending transactions (mempool):
 TXID      vsize    fee (sat)  rate (sat/vB)
 tx1         250         5000         20.00
 tx2         400         4000         10.00
 tx3         150         6000         40.00
 tx4         300         9000         30.00
 tx5         500         7500         15.00
 tx6         200         1000          5.00
 tx7         350        12000         34.29
 tx8         100         2500         25.00

Selected for block (highest fee rate first):
 TXID      vsize    fee (sat)  rate (sat/vB)
 tx3         150         6000         40.00
 tx7         350        12000         34.29
 tx4         300         9000         30.00
 tx8         100         2500         25.00

Left in mempool (did not fit):
 TXID      vsize    fee (sat)  rate (sat/vB)
```

tx1	250	5000	20.00
tx5	500	7500	15.00
tx2	400	4000	10.00
tx6	200	1000	5.00

Block space used : 900 / 1000 vB
Total transaction fees : 29500 sat (0.00029500 BTC)
Block subsidy : 312500000 sat (3.12500000 BTC)
Final miner reward : 312529500 sat (3.12529500 BTC)

[FLAG: The screenshot's table header reads 'TXID' while the code above prints "TxID" - the PDF itself is inconsistent (the screenshot may come from a slightly different build). Both are reproduced exactly as given. Column spacing in the transcription is approximate.]

Q2. Bitcoin Block Validation Module (Rust)

EXACT QUESTION:
"Implement a Bitcoin block validation module using Rust. The program should verify whether a mined block is valid by checking the previous block hash, Merkle Root, block hash, and Proof-of-Work condition. Modify one block parameter and demonstrate that the block is rejected."

STEP 1: Theory

- Block structure. A Bitcoin block has two parts: a header and a list of transactions. The header contains the version, the hash of the previous block, the Merkle root, the timestamp, the difficulty target, and the nonce. Linking each block to the previous block's hash is what forms the chain.

- Hashing. Bitcoin uses double SHA-256, i.e. SHA256(SHA256(data)). SHA-256 shows the avalanche effect: changing even one bit of the input changes the output completely and unpredictably. The block hash is the double SHA-256 of the block header and acts as the block's identity.

- Merkle tree and Merkle root. Each transaction is hashed to get its transaction ID (leaf). Leaves are then paired and hashed together, level by level (if a level has an odd number of nodes, the last hash is duplicated), until a single hash, the Merkle root, remains. The root is stored in the header, so the header commits to every transaction in the block. Changing any transaction changes its leaf, which changes the Merkle root all the way up.

- Proof-of-Work (PoW). To add a block, a miner must find a nonce such that the block hash is below the difficulty target (in our implementation: at least 20 leading zero bits). Since hashes are unpredictable, the miner can only try nonces repeatedly (about 2^20, approx. 1 million attempts on average). Mining is expensive, but verifying PoW needs only a single hash computation. This asymmetry is what makes the system secure.

- Block validation. A node never trusts a received block. It independently recomputes and checks the following:

  Check 1 - Previous block hash matches our chain tip
    Proves: Block correctly extends the chain
  Check 2 - Merkle root recomputed from transactions equals header's Merkle root
    Proves: Transactions are unmodified
  Check 3 - Double SHA-256 of header equals the claimed block hash
    Proves: Header is unmodified
  Check 4 - Block hash satisfies the PoW condition
    Proves: Required work was actually done

  If any check fails, the block is rejected. (A full Bitcoin node also verifies signatures, coinbase value, block size and timestamp rules. This module implements the four required checks.)

- Why tampering is detected. Changing a transaction breaks check 2. Changing any header field (nonce, timestamp, previous hash) changes the recomputed hash, breaking check 3 and almost certainly check 4. An attacker would need to redo the Proof-of-Work for that block and every block after it, which is computationally impractical.

STEP 2: Algorithm

A. Mining (to create the block)

1. Compute the Merkle root of the transactions.
2. Build the header with version, previous block hash, Merkle root, timestamp, difficulty, and nonce = 0.
3. Compute the double SHA-256 of the header.
4. If the hash has at least 20 leading zero bits, stop. This is the valid block hash.
5. Otherwise increment the nonce and go to step 3.

B. Validation (main module)

Input : received block, hash of the previous block in our chain
Output: ACCEPTED or REJECTED

1. Check previous hash: if block.header.prev_hash != expected_prev_hash, mark FAIL.
2. Check Merkle root: recompute the Merkle root from the block's transactions. If it differs from block.header.merkle_root, mark FAIL.
3. Check block hash: recompute the double SHA-256 of the header. If it differs from block.hash, mark FAIL.
4. Check Proof-of-Work: count the leading zero bits of the recomputed hash. If it is less than the required difficulty, mark FAIL.
5. If all four checks passed, output ACCEPTED, otherwise output REJECTED.

C. Tamper demonstration

1. Mine and validate an honest block (accepted).
2. Make a copy of the block and change exactly one parameter (a transaction amount, the nonce, the previous hash, or the timestamp).
3. Run the validator on the modified block and show which check fails and that the block is rejected.

STEP 3: Code (Rust)

```
use sha2::{Digest, Sha256};

/// Difficulty = number of leading zero BITS required in the block hash.
/// (Real Bitcoin encodes a 256-bit target in `nBits`; leading zero bits is the same idea, simplified.)
const DIFFICULTY_BITS: u32 = 20;

type Hash = [u8; 32];

fn sha256d(data: &[u8]) -> Hash {
    let first = Sha256::digest(data);
    let second = Sha256::digest(first);
    second.into()
}

fn hex(h: &Hash) -> String {
    h.iter().map(|b| format!("{:02x}", b)).collect()
}

fn leading_zero_bits(h: &Hash) -> u32 {
    let mut count = 0;
    for byte in h {
        if *byte == 0 {
            count += 8;
        } else {
            count += byte.leading_zeros();
            break;
        }
    }
    count
}

/// Transaction id = double SHA-256 of the transaction data.
fn txid(tx: &str) -> Hash {
    sha256d(tx.as_bytes())
}

/// Bitcoin-style Merkle root: pair up hashes, hash each pair, duplicate the last if the count is odd.
fn merkle_root(txs: &[String]) -> Hash {
    if txs.is_empty() {
        return [0u8; 32];
    }
    let mut level: Vec<Hash> = txs.iter().map(|t| txid(t)).collect();
```

```
    while level.len() > 1 {
        if level.len() % 2 == 1 {
            level.push(*level.last().unwrap());
        }
        level = level
            .chunks(2)
            .map(|pair| {
                let mut buf = Vec::with_capacity(64);
                buf.extend_from_slice(&pair[0]);
                buf.extend_from_slice(&pair[1]);
                sha256d(&buf)
            })
            .collect();
    }
    level[0]
}

#[derive(Clone, Debug)]
struct BlockHeader {
    version: u32,
    prev_hash: Hash,
    merkle_root: Hash,
    timestamp: u32,
    difficulty_bits: u32,
    nonce: u32,
}

impl BlockHeader {
    fn serialize(&self) -> Vec<u8> {
        let mut v = Vec::with_capacity(80);
        v.extend_from_slice(&self.version.to_le_bytes());
        v.extend_from_slice(&self.prev_hash);
        v.extend_from_slice(&self.merkle_root);
        v.extend_from_slice(&self.timestamp.to_le_bytes());
        v.extend_from_slice(&self.difficulty_bits.to_le_bytes());
        v.extend_from_slice(&self.nonce.to_le_bytes());
        v
    }

    fn hash(&self) -> Hash {
        sha256d(&self.serialize())
    }
}

#[derive(Clone, Debug)]
struct Block {
    header: BlockHeader,
    hash: Hash, // hash claimed by the miner
    transactions: Vec<String>,
}

impl Block {
```

```
    /// Miner: build a block and search for a nonce that satisfies the PoW condition.
    fn mine(prev_hash: Hash, transactions: Vec<String>, timestamp: u32) -> Block {
        let mut header = BlockHeader {
            version: 1,
            prev_hash,
            merkle_root: merkle_root(&transactions),
            timestamp,
            difficulty_bits: DIFFICULTY_BITS,
            nonce: 0,
        };
        loop {
            let h = header.hash();
            if leading_zero_bits(&h) >= header.difficulty_bits {
                return Block { header, hash: h, transactions };
            }
            header.nonce += 1;
        }
    }
}

struct Check {
    name: &'static str,
    passed: bool,
    detail: String,
}

/// Validator: run all four checks against the expected previous hash.
fn validate_block(block: &Block, expected_prev_hash: &Hash) -> (bool, Vec<Check>) {
    let mut checks = Vec::new();

    // 1. Previous block hash
    let c1 = block.header.prev_hash == *expected_prev_hash;
    checks.push(Check {
        name: "Previous block hash",
        passed: c1,
        detail: format!(
            "header: {}... expected: {}...",
            &hex(&block.header.prev_hash)[..16],
            &hex(expected_prev_hash)[..16]
        ),
    });

    // 2. Merkle root
    let recomputed_root = merkle_root(&block.transactions);
    let c2 = block.header.merkle_root == recomputed_root;
    checks.push(Check {
        name: "Merkle root",
        passed: c2,
        detail: format!(
            "header: {}... recomputed: {}...",
            &hex(&block.header.merkle_root)[..16],
            &hex(&recomputed_root)[..16]
```

```
            ),
        });

        // 3. Block hash
        let recomputed_hash = block.header.hash();
        let c3 = block.hash == recomputed_hash;
        checks.push(Check {
            name: "Block hash",
            passed: c3,
            detail: format!(
                "claimed: {}... recomputed: {}...",
                &hex(&block.hash)[..16],
                &hex(&recomputed_hash)[..16]
            ),
        });

        // 4. Proof-of-Work (checked on the recomputed hash, never on the claimed one)
        let zeros = leading_zero_bits(&recomputed_hash);
        let c4 = zeros >= block.header.difficulty_bits && block.header.difficulty_bits >=
DIFFICULTY_BITS;
        checks.push(Check {
            name: "Proof-of-Work",
            passed: c4,
            detail: format!("leading zero bits: {} (required: {})", zeros, DIFFICULTY_BITS),
        });

        (checks.iter().all(|c| c.passed), checks)
    }

    fn report(title: &str, block: &Block, expected_prev: &Hash) -> bool {
        println!("--- {title} ---");
        let (ok, checks) = validate_block(block, expected_prev);
        for c in &checks {
            println!(
                "  [{}] {:<20} {}",
                if c.passed { "PASS" } else { "FAIL" },
                c.name,
                c.detail
            );
        }
        println!("  => Block {}\n", if ok { "ACCEPTED" } else { "REJECTED" });
        ok
    }

    fn main() {
        println!("=== Bitcoin Block Validation (DA-7 Q2) ===");
        println!("Difficulty: {} leading zero bits\n", DIFFICULTY_BITS);

        // Chain: genesis -> block 1
        let genesis = Block::mine([0u8; 32], vec!["coinbase -> Miner: 3.125 BTC".to_string()],
            1_700_000_000);
        println!("Genesis block mined, hash = {}", hex(&genesis.hash));
```

```
    let txs = vec![
        "coinbase -> Miner: 3.125 BTC".to_string(),
        "Alice -> Bob: 0.50 BTC".to_string(),
        "Bob -> Carol: 0.20 BTC".to_string(),
        "Dave -> Erin: 1.10 BTC".to_string(),
        "Erin -> Frank: 0.05 BTC".to_string(),
    ];
    let block1 = Block::mine(genesis.hash, txs, 1_700_000_600);
    println!(
        "Block 1 mined (nonce = {}), hash = {}\n",
        block1.header.nonce,
        hex(&block1.hash)
    );

    // Valid block
    let valid = report("Test 1: untouched block", &block1, &genesis.hash);
    assert!(valid, "honest block must be accepted");

    // Tamper with ONE parameter at a time on fresh copies of the valid block.

    // (a) Change a transaction amount (Merkle root no longer matches)
    let mut b = block1.clone();
    b.transactions[1] = "Alice -> Bob: 5.00 BTC".to_string();
    assert!(!report("Test 2: transaction data modified (0.50 -> 5.00 BTC)", &b, &genesis.hash));

    // (b) Change the nonce (hash no longer matches, PoW broken)
    let mut b = block1.clone();
    b.header.nonce = b.header.nonce.wrapping_add(1);
    assert!(!report("Test 3: nonce modified", &b, &genesis.hash));

    // (c) Change previous block hash (block does not extend our chain tip)
    let mut b = block1.clone();
    b.header.prev_hash[0] ^= 0x01;
    assert!(!report("Test 4: previous block hash modified", &b, &genesis.hash));

    // (d) Change the timestamp (hash changes)
    let mut b = block1.clone();
    b.header.timestamp += 1;
    assert!(!report("Test 5: timestamp modified", &b, &genesis.hash));

    println!("All tamper tests were rejected, honest block was accepted.");
}
```

[FLAG: Code indentation was lost in the PDF text and has been reconstructed; all tokens, names, strings and comments are unchanged. Line breaks inside a few statements (the genesis Block::mine call, the Check { ... } struct literals, and the 'nonce: 0' field) were scrambled in the PDF text and are re-flowed from the page layout. Column spacing inside format strings is approximate.]

STEP 4: Output (PDF screenshots of the terminal)

[SCREENSHOT 1 - terminal output, Tests 1 to 3]

```
PS C:\Users\kushagra\BDLT_DA5 > cargo run --release --bin q2
=== Bitcoin Block Validation (DA-7 Q2) ===
Difficulty: 20 leading zero bits

Genesis block mined, hash =
000002d22ea7e101318708a094eeab3b08edaf246228d2b75fca9f96584060af
Block 1 mined (nonce = 1062247), hash =
00000f51b871f72ad57fa1e00cb7164d5304d864a3debb3531eb7b64f3c652c2

--- Test 1: untouched block ---
  [PASS] Previous block hash  header: 000002d22ea7e101... expected: 000002d22ea7e101...
  [PASS] Merkle root          header: fd6fc80135f9d71b... recomputed: fd6fc80135f9d71b...
  [PASS] Block hash           claimed: 00000f51b871f72a... recomputed: 00000f51b871f72a...
  [PASS] Proof-of-Work        leading zero bits: 20 (required: 20)
  => Block ACCEPTED

--- Test 2: transaction data modified (0.50 -> 5.00 BTC) ---
  [PASS] Previous block hash  header: 000002d22ea7e101... expected: 000002d22ea7e101...
  [FAIL] Merkle root          header: fd6fc80135f9d71b... recomputed: 65a723dceff0f0df...
  [PASS] Block hash           claimed: 00000f51b871f72a... recomputed: 00000f51b871f72a...
  [PASS] Proof-of-Work        leading zero bits: 20 (required: 20)
  => Block REJECTED

--- Test 3: nonce modified ---
  [PASS] Previous block hash  header: 000002d22ea7e101... expected: 000002d22ea7e101...
  [PASS] Merkle root          header: fd6fc80135f9d71b... recomputed: fd6fc80135f9d71b...
  [FAIL] Block hash           claimed: 00000f51b871f72a... recomputed: ca741996b7f79589...
  [FAIL] Proof-of-Work        leading zero bits: 0 (required: 20)
  => Block REJECTED
```

[SCREENSHOT 2 - terminal output, Tests 4 and 5]

```
--- Test 4: previous block hash modified ---
  [FAIL] Previous block hash  header: 010002d22ea7e101... expected: 000002d22ea7e101...
  [PASS] Merkle root          header: fd6fc80135f9d71b... recomputed: fd6fc80135f9d71b...
  [FAIL] Block hash           claimed: 00000f51b871f72a... recomputed: 3d2c5613242e7cb2...
  [FAIL] Proof-of-Work        leading zero bits: 2 (required: 20)
  => Block REJECTED

--- Test 5: timestamp modified ---
  [PASS] Previous block hash  header: 000002d22ea7e101... expected: 000002d22ea7e101...
  [PASS] Merkle root          header: fd6fc80135f9d71b... recomputed: fd6fc80135f9d71b...
  [FAIL] Block hash           claimed: 00000f51b871f72a... recomputed: 9b5caa2e68e76054...
  [FAIL] Proof-of-Work        leading zero bits: 0 (required: 20)
  => Block REJECTED

All tamper tests were rejected, honest block was accepted.
```

[FLAG: Hash values were read from small terminal text; the full genesis and block 1 hashes are 64 hex characters each as transcribed, and the shorter 16-character prefixes were read as shown. Verify against the PDF before relying on any single character. Column spacing is approximate. A partially cut-off line

is visible below the last message in the second screenshot and is
not legible, so it is not transcribed.]

FLAGS FOR SUBMISSION 7 (unreadable / ambiguous items):
- Q1: screenshot header 'TXID' vs code string "TxID" (inconsistent in the PDF itself).
- Q1/Q2: code indentation reconstructed; screenshot column spacing approximate.
- Q2: small hash text in screenshots; cut-off line under the final message not legible.

==========
END OF APPENDED SUBMISSION (LAB SUBMISSION 7)
==========

##########
ADDITIONAL QUESTIONS (Q1, Q3 AND Q4)
##########

Note: question numbers are kept exactly as given (Q2 was not supplied).
Q1 uses the same Geth dev-mode procedure as Assignment 2, Q2 above,
written out again in full as requested.

Q1. Gas Estimation and Transaction Execution Using Geth

EXACT QUESTION:
"Start Geth in Development Mode and use eth.estimateGas() to determine the gas required to transfer 2 ETH from the default Ethereum account to itself. Execute the transaction using a gas limit lower than the estimated value and record the error message generated by Geth. Without modifying the sender account, receiver account, or Ether value, repeat the transaction using the exact estimated gas value. Retrieve the transaction details, transaction receipt, and the block containing the successful transaction. Finally, compare the estimated gas with the actual gas used in the transaction receipt and explain the reason for any difference."

STEP 1: Start Geth in dev mode
```
geth --dev console
```

(Dev mode creates one pre-funded account and mines a block automatically whenever a transaction is sent)

STEP 2: Check the default account (sender = receiver)
```
eth.accounts[0]
```

STEP 3: Estimate gas required for the transfer

```
   eth.estimateGas({
      from: eth.accounts[0],
      to: eth.accounts[0],
      value: web3.toWei(2, "ether")
   })

   Output: 21000

STEP 4: Attempt the transaction with a LOWER gas limit than estimated
   eth.sendTransaction({
      from: eth.accounts[0],
      to: eth.accounts[0],
      value: web3.toWei(2, "ether"),
      gas: 20000
   })

   Expected error (record this exactly):
   Error: intrinsic gas too low: gas 20000, minimum needed 21000

   (Wording depends on the Geth version; newer versions print
   "intrinsic gas too low: have 20000, want 21000". Record whatever
   your Geth prints.)

STEP 5: Repeat the SAME transaction using the exact estimated gas
(same sender, same receiver, same value -- only gas changes)
   eth.sendTransaction({
      from: eth.accounts[0],
      to: eth.accounts[0],
      value: web3.toWei(2, "ether"),
      gas: 21000
   })

   Output: transaction hash, e.g.
   "0x967b64ad426d2902444f2852237c21838f65cb3d5dd529020145d38119e71aca"

STEP 6: Retrieve transaction details (use your actual tx hash)
   eth.getTransaction("<paste_tx_hash_here>")

   Note: gas: 21000, gasPrice, from, to, value, nonce, blockNumber

STEP 7: Retrieve the transaction receipt
   eth.getTransactionReceipt("<paste_tx_hash_here>")

   Note: status: "0x1" (success), gasUsed: 21000,
   cumulativeGasUsed: 21000, blockNumber

STEP 8: Retrieve the block containing the successful transaction
   eth.getBlock(eth.blockNumber)

   Note: gasUsed, transactions array containing your tx hash,
   miner address, timestamp
```

STEP 9: Compare estimated gas with actual gas used
eth.getTransactionReceipt("<paste_tx_hash_here>").gasUsed

Output: 21000 (same as the estimateGas() result of 21000)

EXPLANATION TO GIVE:
- estimateGas() returned 21000, and the actual gasUsed in the receipt was also exactly 21000. There is no difference.
- The first attempt with gas = 20000 failed immediately because it is below the 21000 minimum (intrinsic gas) that every transaction must pay; the transaction was rejected before being mined, so nothing was spent and no transaction hash was created.
- A plain ETH transfer (no contract code, no calldata) always costs exactly the base intrinsic gas of 21000 -- a fixed protocol constant -- so the estimate and the actual usage match exactly. Sending 2 ETH from an account to itself changes nothing else, so no extra gas is used.
- If this had been a contract call, estimated and actual gas could differ because the estimate assumes the worst-case path, while actual execution may skip some branches (if/require) and receive gas refunds (e.g. clearing storage).

Q3. Blockchain-Based Digital Wallet System (Solidity, Remix VM)

EXACT QUESTION:
"A company wants to implement a blockchain-based digital wallet system for its employees to securely manage digital funds. The system should maintain individual employee wallet balances on the Ethereum blockchain. Employees should be able to add funds to their wallets, withdraw funds when required, transfer funds to other employee wallets, and verify their available balance. The system should ensure that invalid transactions, such as withdrawing more funds than the available balance, are automatically rejected."

"Develop a Solidity smart contract and deploy it on the Ethereum Virtual Machine (EVM). Compile and test the contract using a suitable Ethereum testing environment."

STEP 1: Open https://remix.ethereum.org

STEP 2: In File Explorer, create a new file: Ques3.sol

STEP 3: Paste the following code exactly

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract EmployeeWallet {
```

```
// Wallet balance of every employee (stored in wei)
mapping(address => uint256) private balances;

event FundsAdded(address indexed employee, uint256 amount);
event FundsWithdrawn(address indexed employee, uint256 amount);
event FundsTransferred(address indexed from, address indexed to, uint256 amount);

// Add funds to the caller's wallet
function addFunds() public payable {
    require(msg.value > 0, "Amount must be greater than zero.");

    balances[msg.sender] += msg.value;

    emit FundsAdded(msg.sender, msg.value);
}

// Withdraw funds from the caller's wallet
function withdrawFunds(uint256 amount) public {
    require(amount > 0, "Amount must be greater than zero.");
    require(balances[msg.sender] >= amount, "Insufficient balance.");

    // Update the balance BEFORE sending Ether (prevents re-entrancy)
    balances[msg.sender] -= amount;

    (bool sent, ) = payable(msg.sender).call{value: amount}("");
    require(sent, "Withdrawal failed.");

    emit FundsWithdrawn(msg.sender, amount);
}

// Transfer funds to another employee's wallet
function transferFunds(address to, uint256 amount) public {
    require(to != address(0), "Invalid recipient address.");
    require(to != msg.sender, "Cannot transfer to yourself.");
    require(amount > 0, "Amount must be greater than zero.");
    require(balances[msg.sender] >= amount, "Insufficient balance.");

    balances[msg.sender] -= amount;
    balances[to] += amount;

    emit FundsTransferred(msg.sender, to, amount);
}

// Verify the caller's available balance
function checkBalance() public view returns (uint256) {
    return balances[msg.sender];
}

// Verify the balance of any employee wallet
function getBalance(address employee) public view returns (uint256) {
    return balances[employee];
}
```

}

STEP 4: Go to "Solidity Compiler" tab (left sidebar)
- Compiler version should auto-match ^0.8.20 (pick any 0.8.x)
- Click "Compile Ques3.sol"
- Confirm green checkmark / "Compiled" status

STEP 5: Go to "Deploy & Run Transactions" tab
- Environment dropdown -> select "Remix VM (Cancun)"
- Contract dropdown -> select "EmployeeWallet"
- Leave VALUE = 0 and click "Deploy"
- Contract appears under "Deployed Contracts"
- Account 1 and Account 2 below show 100 ETH each (note their addresses; use the copy icon next to the Account dropdown)

STEP 6: Add funds from Account 1 (5 ETH)
- Account 1 selected. Set VALUE = 5 and change the unit to "Ether"
- Expand the contract -> click the orange "addFunds" button
- Console log: "Transaction mined and execution completed", status = true
- Set VALUE back to 0 (so it is not sent with later calls)
- Click "checkBalance" -> returns 5000000000000000000 (5 ETH in wei)

STEP 7: Transfer funds from Account 1 to Account 2 (2 ETH)
- Paste Account 2's address into the "to" field of transferFunds
- amount = 2000000000000000000 (2 ETH in wei)
- Click "transact" -> success
- "checkBalance" on Account 1 -> 3000000000000000000

STEP 8: Verify Account 2's balance
- Switch the Account dropdown to Account 2
- Click "checkBalance" -> 2000000000000000000 (or from any account: getBalance(<Account 2 address>))

STEP 9: Withdraw funds (Account 1, 1 ETH)
- Switch back to Account 1
- withdrawFunds(1000000000000000000) -> "transact" -> success
- "checkBalance" -> 2000000000000000000
- Account 1's ETH balance in the Account dropdown increases by about 1 ETH (minus a small gas fee)

STEP 10: Validate invalid withdrawal (more than balance)
- Account 1 has 2 ETH in the wallet
- withdrawFunds(10000000000000000000) (10 ETH) -> "transact"
- Transaction should revert with: "Insufficient balance." (shown in the console error)
- Balance remains unchanged

STEP 11: Validate invalid transfer (more than balance)
- Switch to Account 2 (has 2 ETH in the wallet)
- transferFunds(<Account 1 address>, 5000000000000000000)
- Transaction should revert with:

"Insufficient balance."

STEP 12: Other rejected cases (optional demo)
- addFunds with VALUE = 0 -> "Amount must be greater than zero."
- transferFunds to your own address -> "Cannot transfer to yourself."
- transferFunds to 0x0000000000000000000000000000000000000000 -> "Invalid recipient address."

KEY POINTS TO EXPLAIN IF ASKED:
- mapping(address => uint256) stores each employee's balance on the blockchain, giving O(1) lookup per wallet.
- addFunds() is payable, so Ether sent with the call (msg.value) is credited to msg.sender's balance.
- require() reverts the whole transaction (state changes are undone and unused gas is refunded) when a condition fails -- this is how invalid transactions such as over-withdrawal are automatically rejected.
- withdrawFunds() reduces the balance before sending Ether (checks-effects-interactions) to prevent re-entrancy attacks.
- Solidity 0.8.x has built-in overflow/underflow checks, so balances cannot wrap around.
- Events (FundsAdded, FundsWithdrawn, FundsTransferred) record every operation in the transaction logs.
- Remix VM (Cancun) is the in-browser EVM test environment used to compile, deploy and test the contract.

Q4. Create a Local Ethereum Network Directory and Ethereum Account (Geth)

EXACT QUESTION:
"Create a local Ethereum network directory and an Ethereum account."

STEP 1: Check that Geth is installed
```
geth version
```

STEP 2: Create the local network directory
```
mkdir -p ~/private-eth/node1
cd ~/private-eth
```

STEP 3: Create an Ethereum account inside this directory
```
geth account new --datadir ./node1
```

- Enter a password when asked and repeat it to confirm
- Output shows:
  Public address of the key: 0x<your 40-hex-digit address>
  Path of the secret key file: node1/keystore/UTC--<timestamp>--<address>
- Note down the public address (needed in the next steps)
- Remember the password; the key file cannot be used without it

(Older Geth versions use: geth --datadir ./node1 account new)

STEP 4: List the accounts stored in the directory
geth account list --datadir ./node1

Output: Account #0: {<your address>} keystore://node1/keystore/UTC--...

STEP 5: Create the genesis file
nano genesis.json

Paste the following (replace <YOUR_ADDRESS> with the address from STEP 3, WITHOUT the 0x prefix):

```
{
  "config": {
    "chainId": 12345,
    "homesteadBlock": 0,
    "eip150Block": 0,
    "eip155Block": 0,
    "eip158Block": 0,
    "byzantiumBlock": 0,
    "constantinopleBlock": 0,
    "petersburgBlock": 0,
    "istanbulBlock": 0,
    "berlinBlock": 0,
    "londonBlock": 0
  },
  "difficulty": "1",
  "gasLimit": "8000000",
  "alloc": {
    "<YOUR_ADDRESS>": { "balance": "1000000000000000000000" }
  }
}
```

(balance is in wei: 1000 ETH = 1 followed by 21 zeros)
(NOTE: if your Geth version rejects this genesis because it no longer supports the older consensus settings, add "terminalTotalDifficulty": 0 inside "config". Steps 1-4 and 7 do not depend on the genesis file.)

STEP 6: Initialize the network directory with the genesis block
geth init --datadir ./node1 genesis.json

Expected output includes: "Successfully wrote genesis state"

(Older Geth versions use: geth --datadir ./node1 init genesis.json)

STEP 7: Check the directory structure
ls ./node1

Output: geth keystore

PROJECT STRUCTURE:

```
private-eth/
├── genesis.json
└── node1/
    ├── geth/
    │   └── chaindata/
    └── keystore/
        └── UTC--<timestamp>--<address>
```

STEP 8: Start the node and verify the account in the console

```
geth --datadir ./node1 --networkid 12345 --nodiscover console
```

Inside the console:

```
eth.accounts
```

(Output: ["0x<your address>"])

```
eth.getBalance(eth.accounts[0])
```

(Output: 10000000000000000000)

```
web3.fromWei(eth.getBalance(eth.accounts[0]), "ether")
```

(Output: 1000)

```
net.version
```

(Output: "12345")

```
exit
```

KEY POINTS TO EXPLAIN IF ASKED:

- The data directory (--datadir) holds everything for one node: the blockchain database (geth/chaindata) and the account key files (keystore/).
- geth account new creates a key pair; the account address is derived from the public key, and the private key is stored encrypted (protected by your password) in a JSON file in keystore/.
- genesis.json defines the first block of the private network (chain ID, gas limit and the initial balances in alloc). geth init writes it into the data directory and must be run before starting the node.
- --networkid (12345) matches the chainId in genesis.json; because it is a custom ID, the node cannot join the Ethereum mainnet or public testnets, so the network is private.
- --nodiscover stops the node from looking for other peers, keeping the network local.
- Never share the keystore file or its password; anyone with both can control the account.

==========

END OF ADDITIONAL QUESTIONS (Q1, Q3 AND Q4)

==========